

리눅스 네트워크 프로그래밍

9장. 네트워크 기초



목차

1. TCP/IP 프로토콜의 이해
2. 네트워크 설정 파일과 진단도구

TCP/IP 프로토콜의 이해

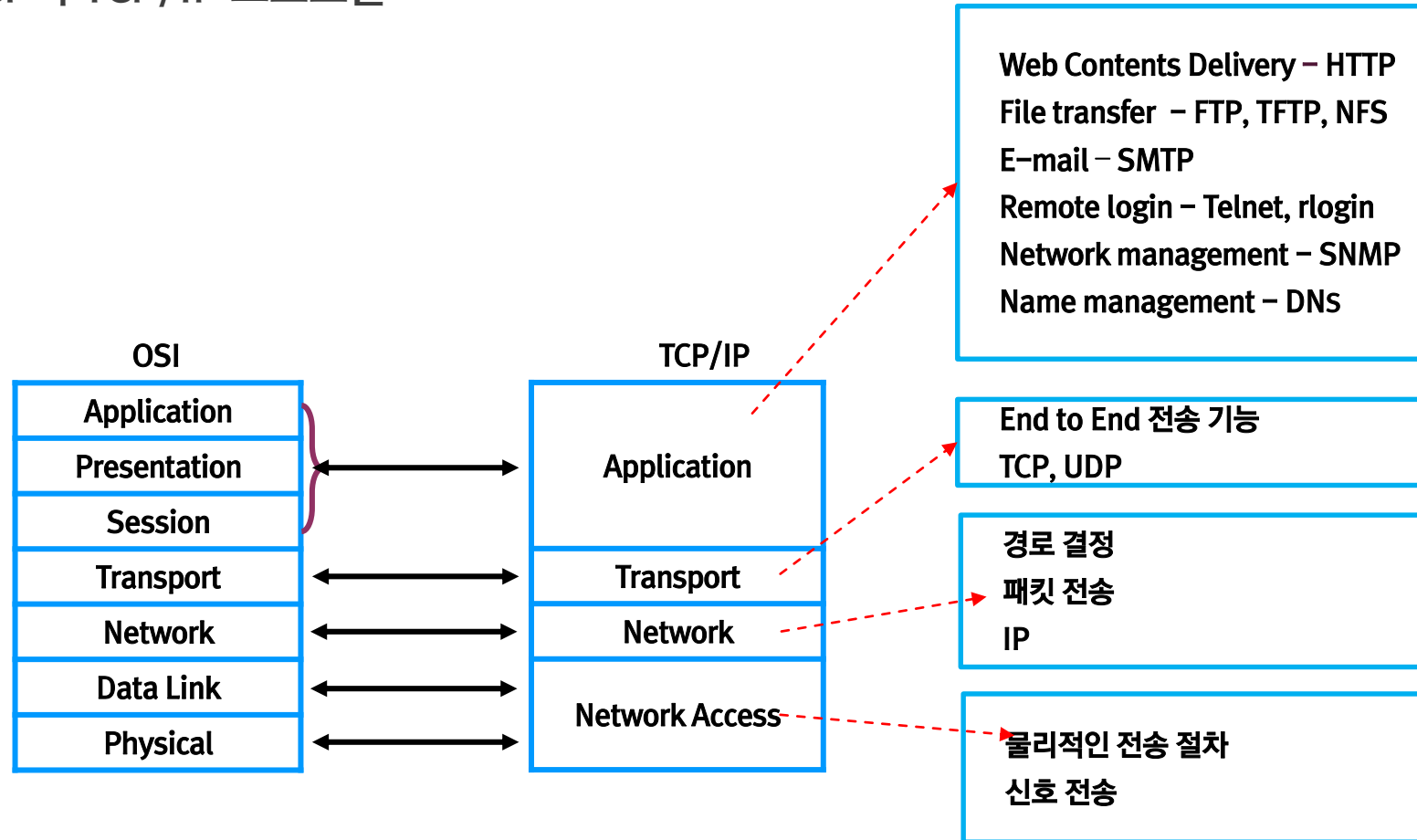
1. 프로토콜이란?
2. TCP/IP 프로토콜 스택

프로토콜이란

- 프로토콜이란 서로 다른 컴퓨터 사이에서 정보를 전송하기 위한 통신 규약으로 사람들이 사용하는 언어와 유사하다. 언어체계가 다른 나라의 언어를 사용하는 사람끼리 대화를 나눌 수 없듯이 컴퓨터 간에도 통신을 위해서는 서로 일정한 형식의 체계가 필요하다.
- 국제표준협회에서는 프로토콜의 표준 모델인 OSI 7 Layer 를 발표하여 각 Layer를 기능별로 독립시키고 각 Layer와 Layer 사이에 상호 연결성과 호환성을 확보하여 Architecture와 OS에 상관 없이 원활한 Data 전송이 가능하도록 표준안을 마련 하였다. 이 표준안에 근거하여 다양한 프로토콜이 개발되었지만 가장 널리 사용되고 있는 프로토콜은 인터넷의 기본 프로토콜인 TCP/IP 프로토콜이다.

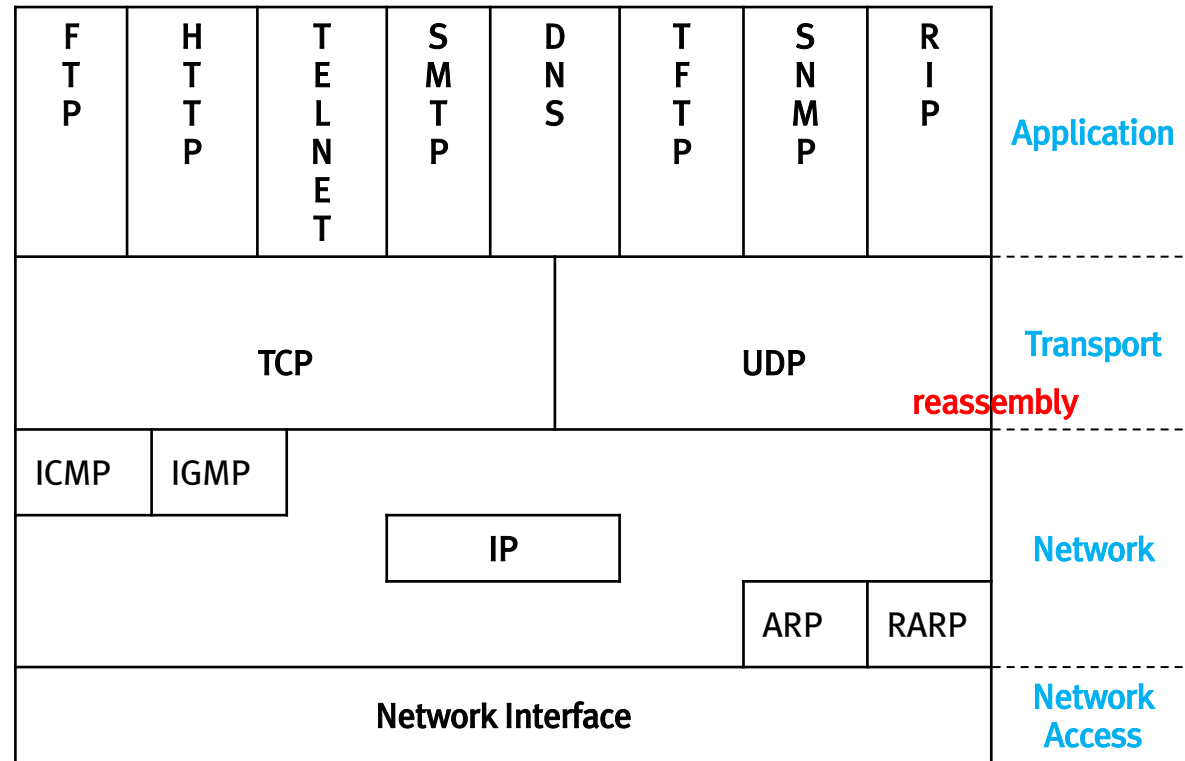
프로토콜이란

■ OSI & Layer 와 TCP/IP 프로토콜



TCP/IP 프로토콜 스택

- TCP/IP 프로토콜 스택과 서비스

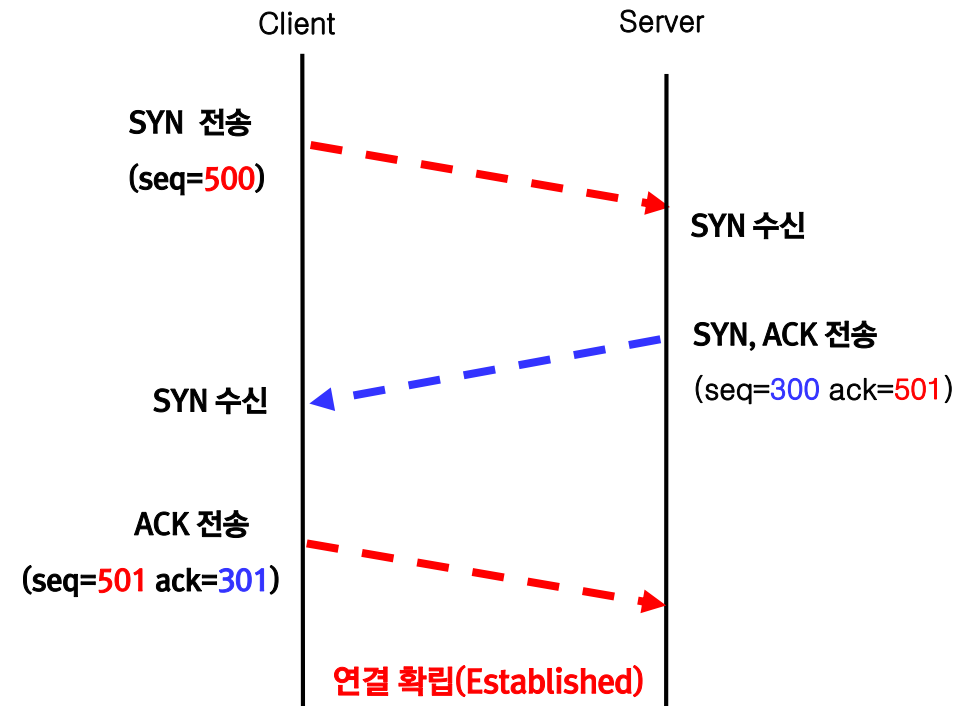


TCP/IP 프로토콜 스택

- TCP(Transmission Control Protocol) 프로토콜
 - Connection-Oriented Protocol
 - 3-Way Handshake
 - Virtual Circuit
 - Data 전송 전에 반드시 논리적인 연결이 수립되어야 함
 - 메시지의 일련번호(Sequence Number)와 응답 번호(Acknowledgement Number)를 이용한 신뢰성 있는 전송
 - Windows Size 이용한 흐름 제어

3-way Handshake

3-WAY HANDSHAKE



TCP/IP 프로토콜 스택

- 다음은 TCP 프로토콜의 헤더 포맷이다.

Source Port Number(16bits)					Destination Port Number(16bits)				
Sequence Number(32bits)									
Acknowledgment Number(32bits)									
Header length	Reserve d (6bist)	U R G	A C K	P U S H	R S T	S Y N	F I N	Window Size(16bits)	
TCP Checksum(16bits)					Urgent Pointer(16bits)				
Option									
Data									

UDP (USER DATAGRAM PROTOCOL)프로토콜

- UDP 프로토콜의 특징
 - Connectionless Protocol
 - Data 전송을 위하여 사전에 필요한 절차가 없음
 - 데이터의 전달을 확인할 수 없어 신뢰성을 보장 못함
 - 응용 계층과의 오버헤드가 없어 신속한 처리가 장점 임.
- 다음은 UDP 프로토콜의 헤더 포맷이다.

Source Port Number(16bits)	Destination Port Number(16bits)
UDP Length (16bits)	UDP Checksum(16bits)
Data	

TCP VS UDP

- 다음은 TCP 와 UDP 프로토콜의 차이점을 정리한 표이다

TCP	UDP
Connection oriented	Connectionless
Sequencing 지원	Sequencing 지원하지 않음
Error control을 한다	Error control 하지 않음
Flow control을 한다	Flow control 하지 않음
Unicast 전송	Unicast, Multicast, Broadcast 전송
Full duplex	Half duplex
데이터 전송	실시간 Traffic 전송 (VoIP, Multimedia 등)

네트워크 설정 파일과 진단 도구

1. 네트워크 설정 파일
2. 네트워크 진단 도구

네트워크 설정 파일

- 다음은 리눅스의 주요 네트워크 설정 파일로 네트워크 프로그램을 실행할 때 관련 정보를 검색하거나 구성 정보로 이용하게 된다.

- /etc/hostname

내 호스트 명이 저장되어 있다.

- /etc/hosts

호스트 명과 IP 주소에 대한 테이블이 등록되어 있다.

```
user@ubuntu: ~  
user@ubuntu:~$ cat /etc/hostname  
ubuntu linux.com  
user@ubuntu:~$  
user@ubuntu:~$  
user@ubuntu:~$ grep 'ubuntu linux.com' /etc/hosts  
192.168.230.149 ubuntu linux.com ubuntu  
user@ubuntu:~$
```

네트워크 설정 파일

- `/etc/resolv.conf`

DNS 서버의 IP 주소가 등록되어있다.

```
user@ubuntu: ~  
user@ubuntu:~$ cat /etc/resolv.conf  
nameserver 168.126.63.1  
search linux.com  
user@ubuntu:~$
```

- `/etc/services`

TCP 또는 UDP 프로토콜의 각종 서비스 명과 포트 번호가 저장되어 있다.

```
user@ubuntu: ~  
user@ubuntu:~$ cat /etc/services | more  
# Network services, Internet style  
#  
# Note that it is presently the policy of IANA to assign a single well-known  
# port number for both TCP and UDP; hence, officially ports have two entries  
# even if the protocol doesn't support UDP operations.  
#  
# Updated from http://www.iana.org/assignments/port-numbers and other  
# sources like http://www.freebsd.org/cgi/cvsweb.cgi/src/etc/services.  
# New ports will be added on request if they have been officially assigned  
# by IANA and used in the real-world or are needed by a debian package.  
# If you need a huge list of used numbers please install the nmap package.  
  
tcpmux      1/tcp          # TCP port service multiplexe  
echo        7/tcp  
echo        7/udp  
discard     9/tcp          sink null  
discard     9/udp          sink null  
sysstat     11/tcp         users  
daytime     13/tcp  
daytime     13/udp  
netstat     15/tcp  
qotd        17/tcp         quote  
mcp         18/tcp         # message-send protocol
```

네트워크 진단도구

- 네트워크 프로그램 개발하다 보면 네트워크 정보가 올바르게 구성되어 있는지 또는 주고 받는 통신 데이터가 정확하게 전달되는지 확인해야 하는 경우가 종종 있다. 이런 경우 리눅스에서 제공하는 네트워크 진단 도구를 이용하면 개발의 속도를 높이고 양질의 프로그램을 개발 할 수 있다. 이제부터 그 도구들에 대하여 알아보기로 하자.

- 네트워크 인터페이스 정보 확인 및 변경 도구

- ifconfig

ifconfig [인터페이스명] [IP주소] [netmask 서브넷주소] up/down

인터페이스명
MAC 주소
IPV4 주소, IPV6 주소
Subnet Mask 주소
현재 상태 (Running - 동작 중)
Tx/Rx 패킷 수
충돌

- ip <명령>

명령	의미
addr	네트워크 장치에 설정되어있는 IP 프로토콜 정보
route	라우팅 테이블 항목
link	네트워크 장치에 대한 정보

네트워크 진단도구

■ 네트워크 상태 확인 도구 – ss

ss [옵션]

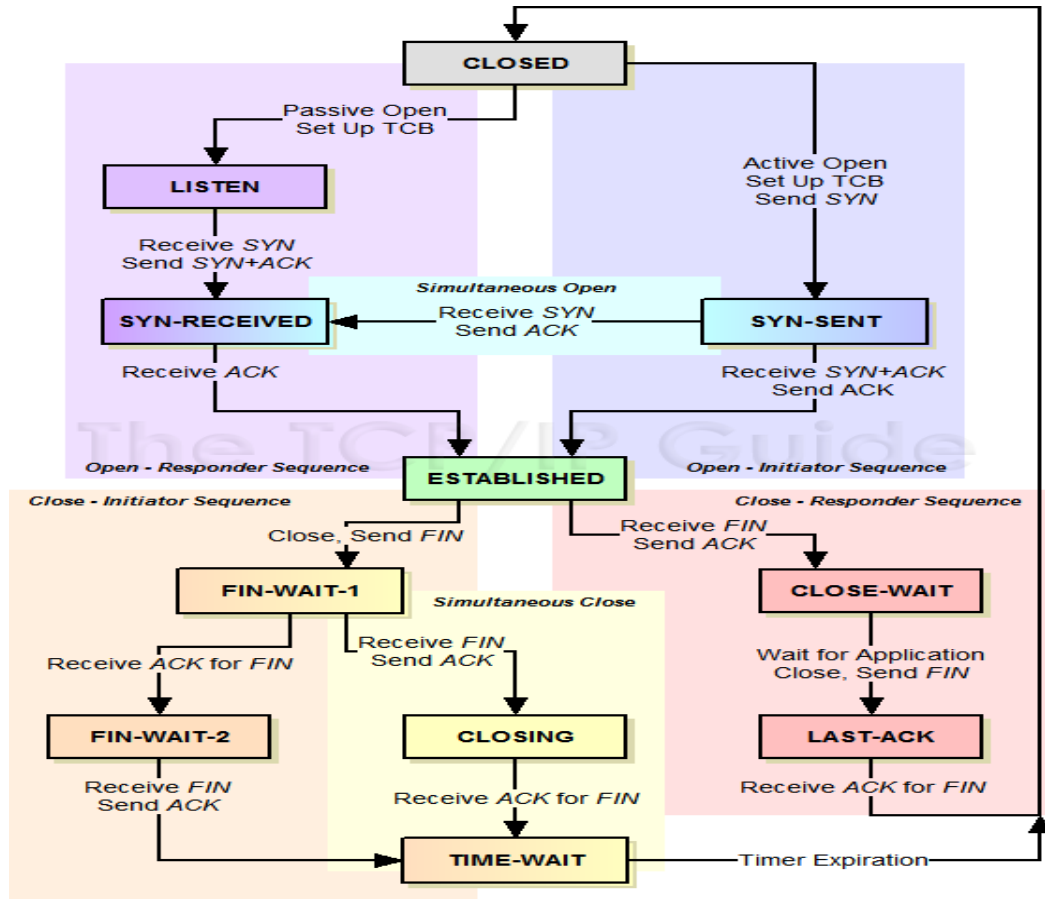
옵션	의미
-an[tu]	활성화되어 있는 네트워크 상태 정보 출력 t : tcp 프로토콜, u : udp 프로토콜
-p	네트워크 서비스와 관련된 프로세스 명 출력
-c	Continue 옵션으로 네트워크 트래픽을 모니터링
-s	네트워크 통계 정보 출력

사용 예1) TCP 프로토콜의 60000번 포트 현재 상태

```
root@ubuntu: ~  
root@ubuntu:~# netstat -antp | grep :60000  
tcp        0      0 0.0.0.0:60000      0.0.0.0:*          LISTEN  
5222/server  
root@ubuntu:~#
```

TCP 상태 천이도

TCP 상태 천이도 (CONT)



상태	의미
LISTEN	서버에서 관련 서비스(데몬)이 서비스가 가능한 상태. 서비스 요청을 기다리고 있는 상태.
SYS_SENT	클라이언트가 서버에게 SYN 패킷을 보낸 후 연결을 요청한 상태.
SYN_RECEIVE D	서버가 클라이언트의 SYN 패킷으로 서비스 요청을 받은 후에 이에 대한 응답으로 SYN/ACK 패킷을 보내고, 클라이언트에게 ACK 패킷을 받기를 기다리는 상태.
ESTABLISHED	서버와 클라이언트간의 3 way handshake(SYN-→SYN/ACK-→ACK) 완료 후에 실제 데이터를 교환하고 있는 상태
FIN_WAIT1 CLOSE_WAIT FIN_WAIT2	연결종료를 위해 종료요청을 받은 후의 종료과정 상태
TIME_WAIT	연결종료 후 일정 시간 동안 유지하고 있는 상태(일정시간 후 자동 종료 됨)
CLOSED	서버와 클라이언트간의 연결이 완전히 종료된 상태

<출처> http://www.tcpipguide.com/free/t_TCPConnectionEstablishmentProcessTheThreeWayHandsh-3.htm

http://www.tcpipguide.com/free/t_TCPConnectionTermination-2.htm

네트워크 진단도구

- 사용 예2) udp 프로토콜의 서비스 상태

```
root@ubuntu: ~  
root@ubuntu:~# netstat -anup | grep udpserver  
udp        0      0 0.0.0.0:60000      0.0.0.0:*  
5504/udpserver  
root@ubuntu:~#
```

- 네트워크 프로그램 작성시 서버 프로그램을 수행하는 경우에는 항상 netstat 도구를 이용하여 서버 프로세스의 네트워크 상태 정보가 정상적으로 수행 중인지 확인하는 것이 좋습니다. 또는 앞서 살펴본 lsof 도구를 이용하여 서버 프로그램이 수행시키는 네트워크 정보가 정상적으로 오픈되었는지 확인하는 방법도 문제 해결에 많은 도움이 됩니다.

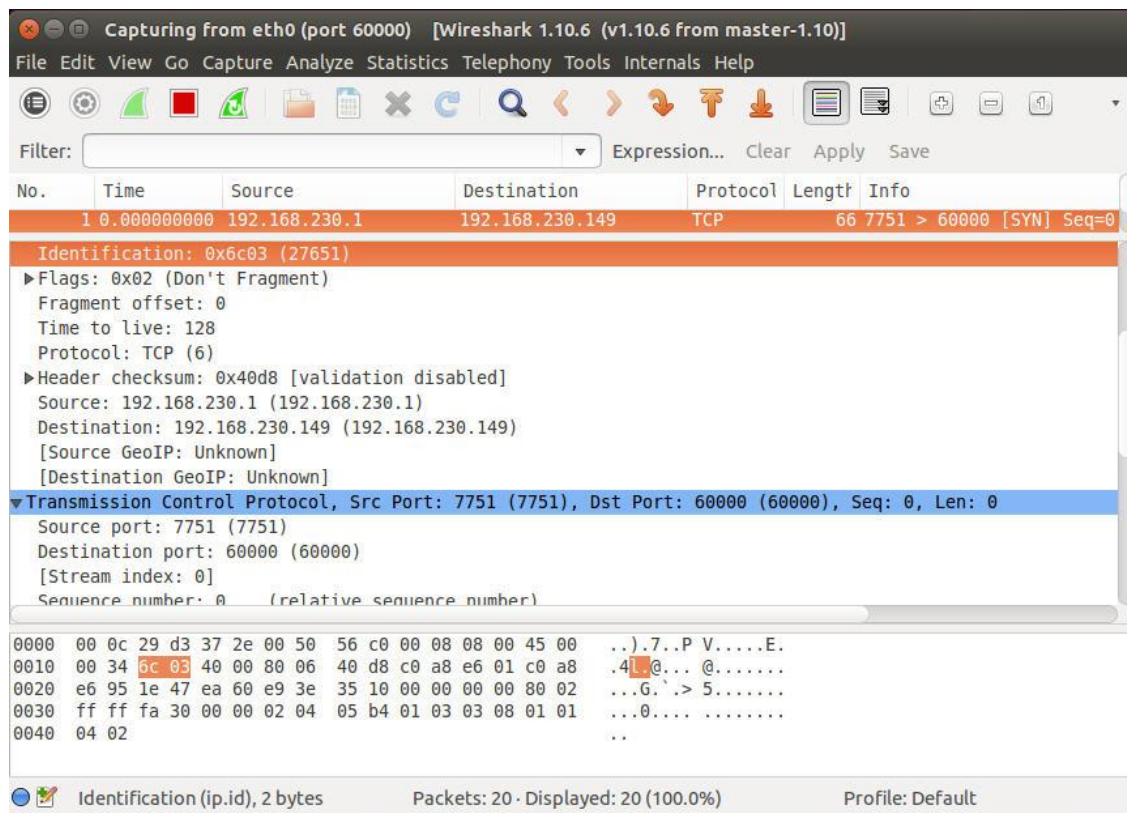
네트워크 진단도구

- 패킷 수집 도구 – tcpdump, wireshark
- 네트워크 상에 주고받는 패킷을 수집하여 헤더 정보와 데이터를 확인하고 분석하는 도구로 tcpdump는 텍스트 기반으로, wireshark는 GUI 기반으로 사용할 수 있다.
- 다음은 tcpdump 도구의 사용 예다.

```
root@ubuntu: ~  
root@ubuntu:~# tcpdump -i eth0 port 60000  
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode  
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes  
00:54:25.700212 IP 192.168.230.1.7670 > ubuntu.linux.com.60000: Flags [S], seq 610292239, win 65535, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0  
00:54:25.700247 IP ubuntu.linux.com.60000 > 192.168.230.1.7670: Flags [S.], seq 4027894338, ack 610292240, win 29200, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0  
00:54:25.700491 IP 192.168.230.1.7670 > ubuntu.linux.com.60000: Flags [.], ack 1, win 1024, length 0  
00:54:25.700565 IP 192.168.230.1.7670 > ubuntu.linux.com.60000: Flags [P.], seq 1:275, ack 1, win 1024, length 274  
00:54:25.700593 IP ubuntu.linux.com.60000 > 192.168.230.1.7670: Flags [.], ack 275, win 237, length 0  
00:54:25.700813 IP ubuntu.linux.com.60000 > 192.168.230.1.7670: Flags [P.], seq 1:19, ack 275, win 237, length 18  
00:54:25.700964 IP 192.168.230.1.7670 > ubuntu.linux.com.60000: Flags [.], ack 19, win 1023, length 0  
00:54:25.701017 IP ubuntu.linux.com.60000 > 192.168.230.1.7670: Flags [P.], seq 19:44, ack 275, win 237, length 25  
00:54:25.701118 IP ubuntu.linux.com.60000 > 192.168.230.1.7670: Flags [FP.], seq 44:127, ack 275, win 237, length 83  
00:54:25.701431 IP 192.168.230.1.7670 > ubuntu.linux.com.60000: Flags [.], ack 128, win 1023, length 0
```

네트워크 진단도구

- 다음은 GUI 환경에서 사용할 수 있는 패킷 수집도구인 wireshak 도구를 이용한 결과다.



실습

- TCP 기반의 서버 프로그램인 msgserver 가 정상적으로 동작되는지 확인하기 위해 테스트하는 중이다. 현재 msgserver 는 수행 중이다. 다음 절차에 적절한 명령 또는 옵션을 입력 하시오.

1) msgserver 프로세스가 60001 번 포트로 동작 중인지 확인해보자.

2) 60001번 포트로 주고 받는 패킷을 수집하여 분석해보려고 한다

리눅스 네트워크 프로그래밍

10장 소켓



목차

1. 소켓
2. 소켓 정보 관련 함수

소켓

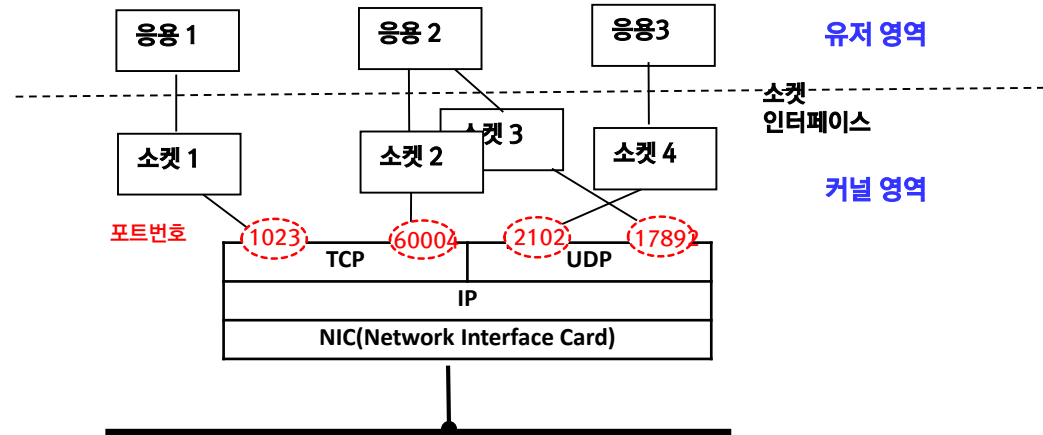
1.소켓이란

2.소켓 주소

소켓이란

■ 소켓의 개념

소켓이란 네트워크를 통해 요구되는 다양한 정보의 송수신 서비스를 위하여 생성되는 논리적인 인터페이스로 TCP/IP 프로토콜을 기반으로 동작된다. 각 소켓에는 IP 주소와 고유한 서비스 번호(포트 번호가) 부여되며 이를 이용하여 클라이언트가 서버로 특정 서비스를 요청하고 그에 대한 서비스를 제공 받을 수 있다. 그림으로 표현하면 다음과 같다.



소켓이란

■ 소켓의 도메인과 유형

- 소켓은 크게 도메인(Domain) 과 유형(Type)으로 구분할 수 있다.
- 여기서 도메인이란 소켓으로 통신이 가능한 영역을 의미한다. 도메인의 종류는 다음과 같다.

도메인 종류	인터페이스	통신 가능 영역	비고
유닉스 도메인	소켓 파일	로컬 내에서만 통신	* 유닉스 계열에서만 지원
인터넷 도메인	IP 주소, 포트번호	원격지와 통신 가능	

- 소켓의 유형은 전송 방식에 관련된 구분으로 다음과 같은 종류가 있다.

소켓 유형	전송방식	데이터 경계	데이터의 순서
스트림 타입	연결 지향 방식	존재하지 않음	보장됨
데이터그램 타입	비 연결 지향 방식	존재함.	보장되지 않음

- 위에 보이는 소켓 유형의 특성에 따르면 스트림 타입은 tcp 프로토콜에, 데이터그램 타입은 udp 프로토콜에 해당하므로 소켓 생성시 이를 기준으로 유형을 선택한다.

소켓주소

- 앞서 그림 에서 살펴보았듯이 응용 프로그램마다 소켓을 이용하여 통신 프로그램을 수행하므로 소켓은 다른 소켓과 구분할 수 있는 주소가 필요한데 다음과 같은 세 종류의 주소가 있다.

도메인 종류	인터페이스	소켓 구조체 자료형	소켓 구조체 자료형
유닉스 도메인	소켓 파일	<pre>struct sockaddr_un { sa_family_t sin_family; char sun_path[108]; };</pre>	<pre>struct sockaddr_un { sa_family_t sin_family; char sun_path[108]; };</pre> 도메인의 종류 지정 - AF_UNIX - AF_INET - AF_INET6
인터넷 도메인	IPV4 주소, 포트번호	<pre>struct sockaddr_in { sa_family_t sin_family; unsigned short int sin_port; struct in_addr sin_addr; char sin_zero[8]; }; struct in_addr { uint32_t s_addr; };</pre>	<pre>struct sockaddr_in { sa_family_t sin_family; unsigned short int sin_port; struct in_addr sin_addr; char sin_zero[8]; }; struct in_addr { uint32_t s_addr; };</pre> 16비트의 포트번호 32비트의 IP 주소 메모리 정렬(align)을 위한 padding 부분
	IPV6 주소, 포트번호	<pre>struct sockaddr_in6 { sa_family_t sin6_family; unsigned short int sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; //128비트 주소 uint32_t sin6_scope_id; }; struct in6_addr { uint8_t s6_addr[16]; };</pre>	<pre>struct sockaddr_in6 { sa_family_t sin6_family; unsigned short int sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; //128비트 주소 uint32_t sin6_scope_id; }; struct in6_addr { uint8_t s6_addr[16]; };</pre>

소켓주소

- 다음은 네트워크 프로그램 중 일부로 소켓 주소의 변수가 사용되는 예이다.

```
struct sockaddr_in server_addr;

    . . .

If ( connect( socksd, (struct sockaddr *) &server_addr,
sizeof(server_addr)==-1) {
    perror( "connect" );
    return 1;
}
```

- 위의 예에서 server_addr 변수는 소켓 주소의 자료형인 struct sockaddr_in 으로 선언된 변수다.
- connect 함수의 원형에 맞추어 두번째 인자로 전달시 struct sockaddr 자료형으로 캐스팅하여 전달한다.
- 이와 같이 네트워크 프로그램 작성시 소켓 주소를 직접 이용하는 경우가 많으므로 각 구조체 안의 멤버들은 그 자료형과 함께 이용 방법들을 잘 익혀두어야 한다.

소켓 정보 관련 함수

1. 바이트 순서 변환 함수
2. IP 주소 변환 함수
3. 소켓 관련 주소 함수
4. 네임 서비스 관련 함수

바이트 순서 변환 함수

- CPU 아키텍처에 따라 데이터를 메모리에 저장하고 읽어오는 순서가 다를 수 있다. 이를 호스트 바이트 순서(host byte order)라고도 한다.
- 여기에는 빅엔디안(big Endian)과 리틀엔디안(Little Endian), 두 가지 방식이 존재하는데
 - 빅엔디안은 메모리의 낮은 번지로부터 데이터를 저장하는 방식이고,
 - 리틀엔디안은 메모리의 높은 번지에서부터 저장하는 방식이다.
- 예를 들어 4바이트 크기의 int 형 변수 num 에 다음과 같이 저장한다고 가정하자.

num = 0x41424344;

을 수행하면 CPU에 따라 다음과 같이 다르게 저장된다.

메모리주소	
0x10003	0x41
0x10002	0x42
0x10001	0x43
0x10000	0x44

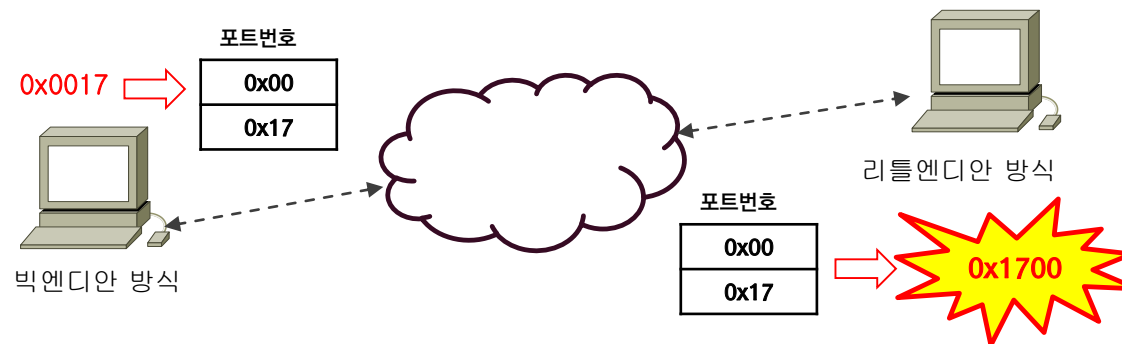
리틀엔디안 방식
(인텔계열)

메모리주소	
0x10003	0x44
0x10002	0x43
0x10001	0x42
0x10000	0x41

빅엔디안 방식
(RISC, SPARC등)

바이트 순서 변환 함수

- 두 엔디안 방식의 장점은 다음과 같다.
 - 빅엔디안 방식은 가독성이 뛰어나고 대소비교가 빠르다
 - 리틀 엔디안 방식은 산술 연산이 빠르다는 장점이 있다.
- 네트워크 프로그램 작성시 이 호스트바이트 순서 문제는 아주 중요한 요소가 된다.
- 예를 들어 빅엔디안 방식의 CPU를 사용하는 컴퓨터에서 리틀엔디안 방식의 CPU를 사용하는 컴퓨터로 포트 번호를 전송한다고 가정해보자.



바이트 순서 변환 함수

- 이와 같은 문제점 때문에 네트워크를 통해 데이터를 송수신할 때 바이트의 순서를 통일하기로 약속을 했고 이를 ‘네트워크 바이트 순서’라고 한다. 이에 따라 네트워크 바이트 순서에 맞추어 변환하는 함수를 다음과 같이 제공하고 있다.

`#include <arpa/inet.h>`

`unsigned short htons(unsigned short)`

short 자료형의 데이터를 호스트 바이트 순서에서 네트워크 바이트 순서로 변환 하는 함수

`unsigned short ntohs(unsigned short)`

short 자료형의 데이터를 네트워크 바이트 순서에서 호스트 바이트 순서로 변환 하는 함수

`unsigned long htonl(unsigned long)`

long 자료형의 데이터를 호스트 바이트 순서에서 네트워크 바이트 순서로 변환 하는 함수

`unsigned long ntohl(unsigned long)`

long 자료형의 데이터를 네트워크 바이트 순서에서 호스트 바이트 순서로 변환 하는 함수

다음 ex01.c 는 바이트 순서를 변환하는 함수에 대한 예제이다 실행해보자

소스코드 (ex01.c)

```
#include <stdio.h>
#include <arpa/inet.h>
int main()
{
    union u_short {
        unsigned short s;
        char s_array[2];
    } port;
    union u_long {
        unsigned long l;
        char l_array[4];
    } addr;
    int i;

    port.s=0x0017;
    addr.l=0x01020304;
    printf("<short> 호스트 바이트 순서 : \n");
    for(i=1; i>=0; i--)
        printf("%p\t%02x\n", &port.s_array[i], port.s_array[i]);
    port.s=htons(port.s);
    printf("<short> 네트워크 바이트 순서 : \n");
    for(i=1; i>=0; i--)
        printf("%p\t%02x\n", &port.s_array[i], port.s_array[i]);
    printf("<long> 호스트 바이트 순서 : \n");
    for(i=3; i>=0; i--)
        printf("%p\t%02x\n", &addr.l_array[i], addr.l_array[i]);
    addr.l=htonl(addr.l);
    printf("<long> 네트워크 바이트 순서 : \n");
    for(i=3; i>=0; i--)
        printf("%p\t%02x\n", &addr.l_array[i], addr.l_array[i]);
    return 0;
}
```

IP 주소 변환 함수

- 문자열/32비트 변환 함수
- 우리가 일반적으로 사용하는 IP 주소는 IPV4, 32비트 주소 체계다. 32비트의 주소를 그대로 표현하면 사람들이 기억하기 쉽지 않기 때문에 8비트 단위로 잘라 10진수 4자리의 문자열로 사용하는 것이 일반적이다. 4자리의 10진수 문자열로 표현된 IP 주소를 32비트의 주소로, 32비트의 주소를 10진수의 4자리 문자열로 변환을 도와주는 함수는 다음과 같다.

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

```
in_addr_t inet_addr(const char *cp)
```

10진수 4자리로 구성된 문자열의 IP 주소를 32비트의 자료형으로 변환하여 반환하는 함수

```
char *inet_ntoa(struct in_addr in)
```

32비트의 주소를 10진수 4자리의 문자열로 변환하는 함수

```
char *inet_aton(const char *cp, struct in_addr *inp)
```

10진수 4자리로 구성된 문자열의 IP 주소를 32비트의 자료형으로 변환하여
두번째 인자인 inp로 반환하는 함수

IPv6 변환 함수

IPV6 변환 함수

- 앞서 소개된 세 함수는 IPv4 전용의 변환 함수다. 만약 IPv6 의 128비트 주소 변환도 고려해야 한다면 다음과 같은 함수를 사용해야 한다.
- `int inet_pton(int address_family, const char* src, void* dst)`
문자열로 구성된 IP 주소를 32비트 또는 128비트로 변환하는 함수로
첫번째 인수는 도메인 종류 AF_INET(IPv4) 또는 AF_INET6(IPv6)를,
두번째 인수 src에는 문자열로 구성된 IP 주소를,
세번째 인자에는 숫자로 변환되는 주소를 받아올 메모리의 시작 주소를 명시한다.
- `const char* inet_ntop(int address_family, const void* src, char* dst, socklen_t dstBytes)`
inet_pton 함수와 반대로 숫자형식의 주소를 문자열 형식으로 변환하는 함수
첫번째 인수는 도메인 종류 AF_INET(IPv4) 또는 AF_INET6(IPv6)를,
두번째 인수 src에는 숫자 형식의 주소를,
세번째 인자에는 문자열로 숫자로 변환되는 주소를 받아올 메모리의 시작 주소를 명시한다.



ex02.c 는 IP 주소 변환하는 함수에 대한 예제
이다 실행 결과를 확인해보자

소켓 관련 주소 함수

- 현재 사용 중인 소켓에 대한 정보를 필요로 하는 경우에는 다음과 같은 함수를 이용한다.

```
#include <sys/socket.h>
```

```
int getsockname(int s, struct sockaddr *name, socklen_t *namelen);
```

이 함수는 지정한 소켓 디스크립터 *s* 에 대한 로컬 정보를 두번째 인자 *name*으로, 소켓 주소의 길이는 세번째 인자 *namelen*으로 반환한다.

```
int getpeername(int s, struct sockaddr *name, socklen_t *namelen);
```

이 함수는 소켓 디스크립터 *s* 에 현재 네트워크로 연결된 호스트의 정보를 알아오는 함수로 호스트의 정보는 두번째 인자인 *name*으로, 소켓 주소의 길이는 세번째 인자인 *namelen*으로 반환한다.

ex03.c 에 현재 연결된 소켓에 대한 정보를 알아보는 함수를 구현하고 실행해보자

소스코드 (ex03.c)

```
•
•
42     client_sockfd = accept(server_sockfd, (struct sockaddr *)&clientaddr,
43                           &client_len);
44     if (client_sockfd == -1)
45     {
46         perror("Accept error : ");
47         exit(0);
48     }
49
50     getsockname(client_sockfd, (struct sockaddr *)&myaddr, &client_len);
51
52     printf("local Port   : %d\n", ntohs(myaddr.sin_port));
53     printf("local address : %s\n", inet_ntoa(myaddr.sin_addr));
54
55     getpeername(client_sockfd, (struct sockaddr *)&myaddr, &client_len);
56
57     printf("client port : %d\n", myaddr.sin_port);
58     printf("client address : %s\n", inet_ntoa(myaddr.sin_addr));
59     close(client_sockfd);
60     return 1;
61 }
```

네임 서비스 관련 함수

- 일반 사용자들이 숫자 형태로 구성되어 있는 IP 주소를 기억해서 통신한다는 것은 매우 불편한 일이다, 그래서 인터넷에는 도메인 네임에 대한 IP 주소를 검색하는 도메인 네임서버를 구축하여 사용하고 있다. 네트워크 프로그램에서도 이 기능을 사용할 수 있는데 관련 함수는 다음과 같다.

```
#include <netdb.h>

struct hostent * gethostbyname(const char *name);
```

- gethostbyname() 함수는 주어진 호스트 명에 대한 정보를 알 수 있는 함수로 인자로 호스명을 전달하면 다음과 같은 구조체에 네트워크 주소에 관련된 정보를 반환한다.

```
struct hostent {
    char *h_name;           // 호스트 공식 이름
    char **h_aliases;       // 호스트 얼라이스 명
    int h_addrtype;         // 주소 타입
    int h_legh;             // 주소 길이(바이트)
    char **h_addr_list;     // IP 주소 배열
    #define h_addr h_addr_list[0]
}
```

확장된 네임서비스 관련 함수

IPV6 네임서비스 관련 함수 (CONT)

- 앞서 살펴본 gethostbyname 함수는 IPv4 전용의 네임서비스 함수다. 기존의 코드들은 이 함수를 이용하여 대부분 작성되어 있어 이 함수에 대해서 알아야 하지만 새로 작성하는 경우에는 IPv6 도 수용하는 함수를 알아두는 것이 좋다. 다음은 IPv4, IPv6 모두에서 사용할 수 있는 네임 서비스 관련 함수다.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *node, const char *service, const struct addrinfo *hints, \
                struct addrinfo **res); void freeaddrinfo(struct addrinfo *res);

void freeaddrinfo(struct addrinfo *res);
```

- getaddrinfo 함수는 네트워크의 주소를 검색하는 함수로 내부적으로 동적 메모리를 할당 받으므로 사용이 끝나면 반드시 freeaddrinfo 함수를 이용하여 메모리를 해제해야 한다.

IPV6 네임서비스 관련 함수

- getaddrinfo 함수에서

첫번째 인자에 호스트명이나 IP 주소를, 두번째 인자에 서비스명이나 포트번호를,

세번째 인자에 검색하고자 하는 주소에 대한 세부 조건을 명시하여 호출하면 마지막 인자인 res 로 네트워크 주소에 관련된 정보를 반환한다. 이때 이용되는 구조체 addrinfo 의 자료형은 다음과 같다

```
struct addrinfo {  
    int      ai_flags; // 제어 정보를 위한 flag  
    int      ai_family; // AF_INET, AF_INET6  
    int      ai_socktype; // STREAM, DGRAM  
    int      ai_protocol; // 프로토콜 번호  
    size_t   ai_addrlen; // 소켓 주소 길이  
    struct sockaddr *ai_addr; // 소켓 주소  
    char     *ai_canonname; // 별명  
    struct addrinfo *ai_next; // 다음 노드에 대한 포인터  
};
```

```
memset(&hints, 0x00, sizeof(struct addrinfo));  
hints.ai_family = AF_INET;  
hints.ai_socktype = SOCK_STREAM;  
  
if(getaddrinfo(NULL, argv[1], &hints, &result) != 0) {  
    perror("getaddrinfo");  
    return 1;  
}  
  
for(rp = result ; rp != NULL; rp = rp->ai_next) {  
    printf( "%08x\n" , rp->ai_addr);  
}  
  
freeaddrinfo(result);
```



호스트명을 전달하여 네트워크 주소를 확인하는 `gethostbyname()` 함수를 이용하여 현재 연결된 소켓에 대한 정보를 알아보는 `ex04.c`를 작성해 봅시다.

실습

- 다음과 같은 조건의 소켓 주소를 초기화 하려고 한다. 소스코드의 빈 칸을 완성하시오.

소스코드 (exercise.c)

```
.....

int main(void)
{
    struct hostent *host;
    struct sockaddr_in server_addr;

    if((host = 1 ("ubuntu.linux.com")) == NULL)
    {
        printf("error\n");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = 2 0x0050);
    server_addr.sin_addr = *((struct in_addr *) host->h_addr);
    memset(&(server_addr.sin_zero), '\0', 8);

    printf("[ %s ]\n", 3 (server_addr.sin_addr));
    return 0;
}
```

• IPV4와 IPv6 의 차이점 <출처 : 한국 인터넷진흥원>

구분	IPv4	IPv6
주소길이	32비트	128비트
표시방법	8비트씩 4부분으로 10진수로 표시 예) 202.30.64.22	16비트씩 8부분으로 16진수로 표시 예) 2001:0230:abcd:ffff:0000:0000:ffff:1111
주소개수	약 43억개	약(43억×43억×43억×43억)개(거의 무한대)
주소할당	A, B, C, D 등 클래스 단위의 비순차적 할당 (비효율적)	네트워크 서 규모 및 단말기 수에 따른 순차적 할당 (효율적)
품질제어	베스트 에퍼트 방식으로 품질 보장이 곤란 (타입이나 서비스에 대한 QoS 일부지원)	등급별, 서비스별로 패킷을 구분할 수 있어 품질보장이 용이 (트래픽 클래스, 폴로 라벨에 의한 QoS지원)
보안기능	IPsec 프로토콜 별도 설치	확장기능에서 기본으로 제공
플러그 앤드 플레이	없음	있음(오디오 컨키큐어레이션 가능)
모바일IP	상당히 곤란(비효율적)	용이(효율적)
웹캐스팅	곤란	용이(스코프 필드 증가)

리눅스 네트워크 프로그래밍

11장 TCP 프로그래밍



목차

1. TCP 서버/클라이언트 구조
2. TCP 소켓 API
3. TCP 서버/클라이언트 프로그램

TCP 서버/클라이언트 구조

- TCP는 connection oriented 프로토콜 이므로 서버는 클라이언트의 연결 요청을 기다리는 상태가 되어 있어야 한다. 서버의 상태가 준비되어있다면 클라이언트에서 요청시 연결이 수립되고 비로서 데이터를 송수신할 수 있는 모드가 된다. 데이터 송수신이 완료되면 일반적으로 클라이언트에서 연결을 종료하고 서버는 무한 루프 구조로 반복한다. 이 내용을 소켓 API 와 함께 일련의 흐름으로 나타내보면 다음과 같다.

TCP 서버

TCP 클라이언트

TCP 서버와 클라이언트 함수 호출 순서

TCP 소켓 API

■ 소켓 생성

- 소켓을 사용하기 위해서는 먼저 `socket()` 함수를 이용하여 소켓을 생성해야 하는데 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/socket.h>

int socket (int domain, int type, int protocol);
```

- 첫 번째 인자 `domain`에는 사용하는 도메인의 종류를 지정한다. 도메인의 종류에는 다음과 같은 종류가 있다.

도메인 종류	인터페이스
AF_UNIX	유닉스 도메인 소켓 (로컬 통신)
AF_INET	IPv4 네트워크 도메인 소켓
AF_INET6	IPv6 네트워크 도메인 소켓

- 두 번째 인자인 `type`에는 소켓 유형을 지정한다. 소켓의 유형에는 다음과 같은 종류가 있다.

소켓 유형	관련 프로토콜
SOCK_STREAM	스트림 타입, TCP
SOCK_DGRAM	데이터그램 타입, UDP
SOCK_RAW	RAW 타입, IP

TCP 소켓 API

- 세 번째 인수 protocol에는 프로토콜의 번호를 지정한다. 그러나 프로토콜 번호는 특별히 지정하지않고 일반적으로 0을 설정한다.
- 이 함수 호출이 성공하면 소켓 디스크립터를 반환한다. 실패하면 -1 을 반환한다.

- TCP 소켓 생성에 관한 예제 코드를 살펴보자

```
int sockfd;  
  
sockfd=socket(AF_INET, SOCK_STREAM, 0);  
if(sockfd==-1) {  
    perror( "socket" );  
    exit(1);  
}
```

- 위의 예에서 도메인을 INET 으로 소켓 유형은 STREAM , 즉 tcp 기반으로 소켓을 생성 했다. 성공한 경우에는 소켓 디스크립터를 반환하는데 이는 리눅스의 프로세스 구조에서 살펴보았던 파일 디스크립터이다. 따라서 파일 디스크립터 관련 함수를 소켓에 적용할 수도 있다.

TCP 소켓 API

■ 주소 설정

- 클라이언트에서 서버로 연결 요청 시 서버의 특정 소켓에서 응답을 하려면 IP 주소와 포트 번호를 설정해야 한다. 이때 사용되는 함수가 `bind()` 함수로 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *my_addr,
socklen_t addrlen);
```

- 첫 번째 인자 `sockfd`에는 `socket()` 함수의 반환 값인 소켓 디스크립터를 지정한다.
- 두 번째 인자 `myaddr`에는 소켓 주소의 정보를 가리키는 주소를 지정한다.

도메인 종류	소켓 구조체 자료형
AF_UNIX	struct sockaddr_un
AF_INET	struct sockaddr_in
AF_INET6	struct sockaddr_in6

- 소켓 주소의 자료형이 다르므로 범용 자료형 `stuct sockaddr` 로 캐스팅해야 한다.
- 세 번째 인자 `addrlen`은 소켓 주소의 길이를 지정한다.
- 이 함수의 호출이 성공하면 0을 반환하고, 실패하면 -1을 반환한다.

TCP 소켓 API

- 다음은 bind 함수의 예제 코드이다.

```
struct sockaddr_in myaddr;  
my_addr.sin_family = AF_INET;  
my_addr.sin_port = htons(60000);  
my_addr.sin_addr.s_addr = htonl(INADDR_ANY);  
memset(&(my_addr.sin_zero), '\0', 8);  
  
if(bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr)) == -1)  
{  
    perror("bind");  
    exit(1);  
}
```

- 위의 예에서 IPv4 용의 소켓 주소 자료형인 sockaddr_in 의 변수 myaddr 에 도메인 종류, 포트번호, IP 주소등을 대입했습니다. 여기서 IP 주소에 INADDR_ANY란 0.0.0.0 으로 정의되어 있는 값으로 이렇게 설정하면 소스 코드 수정 없이 어디서든 수행시킬 수 있다. 이 값을 두번째 인자로 전달할 때 범용 자료형인 (struct sockaddr *) 으로 캐스팅하여 전달했다.

TCP 소켓 API

- 백 로그(backlog) 큐 생성

- listen() 함수는 서버에서 클라이언트의 요청을 받아들이기 위해 준비하는 함수로 백 로그 큐를 생성해주는 함수다. 이 백 로그 큐에는 연결을 요청한 클라이언트의 정보가 저장된다.

```
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);
```

- 첫 번째 인수 sockfd에는 소켓 디스크립터를 지정한다.
- 두 번째 인수 backlog 에는 백 로그 큐의 크기를 지정한다.
- 이 함수의 호출이 성공하면 0을 반환하고, 실패하면 -1을 반환한다.

TCP 소켓 API

■ 연결 수립

- `accept()` 함수는 서버에서 클라이언트의 연결을 기다리는 함수로 연결 요청이 들어올 때까지 서버 프로세스는 수행을 멈춘다. 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *client_addr, socklen_t *addrlen);
```

- 첫 번째 인수 `sockfd`에는 소켓 디스크립터를 지정한다.
- 두 번째 인수 `client_addr`에는 연결을 요청한 클라이언트의 소켓 주소를 받아올 공간의 주소를 전달한다.
- 세 번째 인수 `addrlen`에는 클라이언트의 소켓 주소 길이를 받아올 공간의 주소를 전달한다.
- 이 함수가 성공적으로 호출되면 새로운 소켓 디스크립터를 반환한다. 이 새로운 소켓 디스크립터로 클라이언트와 데이터를 송수신한다. `socket()` 함수에서 반환 받은 소켓 디스크립터는 다른 클라이언트의 연결을 대기하는 디스크립터로 사용한다.

TCP 소켓 API

- 다음은 accept 함수의 예제 코드다.

```
while(1) {
    sin_size = sizeof(struct sockaddr_in);

    if((new_fd = accept(sockfd, (struct sockaddr *) &their_addr, &sin_size)) == -1) {
        perror("accept");
        continue;
    }
}
```

- 앞의 예제 코드에서 accept 함수를 호출하면 프로세스는 클라이언트와 연결이 수립될 때까지 일단 수랭을 멈추고 기다린다. 연결이 수립되면 두번째 인자로 클라이언트의 소켓주소를, 세번째 인자로 소켓 주소의 길이를 전달 받고, 클라이언트와 연결되면서 새로 생성된 소켓 디스크립터를 반환 받는다. 새로운 디스크립터인 new_fd 는 클라이언트와 통신용으로 사용하고 기존의 sockfd는 다시 accept 함수 호출시 첫번째 인자로 사용한다. accept 함수를 빠져 나온 후 fork() 나 pthread_create() 함수를 통해 새로운 프로세스나 쓰레드를 생성하여 병행(concurrent) 서버를 구축할 수도 있다.

TCP 소켓 API

■ 연결 요청

- connect() 함수는 클라이언트에서 서버로 연결 요청할 때 사용하는 함수로 그 원형은 다음과 같다.

```
#include <sys/types.h>  
#include <sys/socket.h>
```

```
int connect(int sockfd, const struct sockaddr *serv_addr, socklen_t addrlen);
```

- 첫 번째 인수 sockfd에는 소켓 디스크립터를 지정한다.
- 두 번째 인수 serv_addr에는 서버의 소켓 주소가 들어있는 변수의 주소를 지정한다.
- 세 번째 인수 addrlen 에는 소켓 주소의 길이를 지정한다.
- 함수의 호출이 성공하면 0을, 실패하면 -1을 반환한다.

TCP 소켓 API

- connect() 함수의 예제 코드는 다음과 같다.

```
their_addr.sin_family = AF_INET;
their_addr.sin_port = htons(60000);
their_addr.sin_addr.s_addr = inet_addr( "192.168.1.10" );
memset(&(their_addr.sin_zero), '\0',8);
if(connect(sockfd, (struct sockaddr *)&their_addr, sizeof(struct sockaddr)) == -1) {
    perror("connect");
    exit(1);
}
```

- 위의 예는 서버의 IP 주소 192.168.1.10, 포트번호 60000번으로 연결을 요청하는 부분이다. 연결 가능하면 즉시 반환하고 연결이 불가능한 경우에는 일정시간 (타임아웃) 시간이 지난 후 함수를 빠져 나온다 반환 값을 이용하여 함수의 성공 여부를 알 수 있다.

TCP 소켓 API

■ 데이터 송수신

- 소켓을 이용하여 데이터를 송수신하기 위하여 read(), write() 함수나 send(), recv() 함수를 이용할 수 있다. read(), write() 함수에 대해서는 파일 처리 회차에서 이미 살펴보았기 때문에 생략한다. send(), recv() 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/socket.h>

int send(int sockfd, const void *buf, size_t len, int flag);
int recv(int sockfd, const void *buf, size_t len, int flag);
```

- 나머지 인수는 read(), write() 함수와 동일하므로 설명을 생략하고 네 번째 인수 flag는 데이터의 속성을 제어하는 플래그로 일반적으로는 0으로 표현한다.

send flag	의미
MSG_OOB	긴급한 데이터 전송
MSG_DONTROUTE	패킷 전송 시 라우팅하지 않고 직접 네트워크의 호스트로 전송
recv flag	의미
MSG_PEEK	recv() 함수에서 데이터 수신 시 버퍼에서 지우지 않고 데이터를 확인
MSG_WAITALL	요구한 최대 크기만큼 데이터가 도착할 때까지 기다림
MSG_OOB	긴급한 데이터 수신

TCP 소켓 API

- 이 함수가 성공적으로 호출되면 실지로 전송, 혹은 수신한 바이트 수를 반환하며, 실패하면 -1을 반환한다. errno 변수를 확인하면 오류의 원인을 알 수 있다.
- 데이터 송수신 관련 예제 코드를 살펴보자

```
while(1) {  
    . . .  
    if(send(sockfd, buf, strlen(buf), 0) == -1)  
        perror( "send" );  
    if((numbytes = recv(sockfd, buf, MAXDATASIZE-1, 0)) == -1) {  
        perror("recv");  
        exit(1);  
    }  
    . . .  
}
```

- 위의 예제 프로그램은 send() 함수를 이용하여 소켓 통해 연결된 원격지로 buf 내용을 전송하고, 다시 recv() 함수를 통해 buf 로 수신 받는 프로그램이다. 플래그 부분을 제외하고는 write(), read() 함수와 아주 유사하게 작성된다.

TCP 소켓 API

■ 연결 해제

- 소켓의 연결을 해제하기 위해서는 close() 나 shutdown() 함수를 이용한다.

```
int close(int sd);  
int shutdown(int sd, int how);
```

- close 함수는 일반 파일을 해제하듯이 소켓 디스크립터를 전달해 해제한다. 이 함수를 호출하면 해당 프로세스와 소켓과의 연결만 해제되고 이 소켓을 열어놓은 다른 프로세스가 있는 경우 모든 프로세스가 해제를 해야 실지로 소켓이 종료 처리된다.
- 이에 반해 shutdown 함수는 즉시 소켓을 종료하므로 이 소켓을 열어놓은 프로세스에게 영향을 미치게 된다. 따라서 일반적으로는 close함수로 해제를 한다.
- shutdown() 함수의 두번째 인자 how 에 지정할 수 있는 값은 다음과 같다.

how	소켓 구조체 자료형
SHUT_RD	읽기 채널만 닫기
SHUT_WR	쓰기 채널만 닫기
SHUT_RDWR	소켓 즉시 종료

TCP 서버/클라이언트 프로그램

1. TCP 서버 프로그램
2. TCP 클라이언트 프로그램

TCP 서버 프로그램

- 앞서 학습한 TCP 소켓 API 를 이용하여 다음과 같은 조건의 TCP 서버 프로그램을 작성해보자.

〈조건〉

소켓 주소 – IP 주소(0.0.0.0)

포트번호(60000)

동시에 다수의 클라이언트 연결 가능(병행 서버)

클라이언트 연결 수립되면 서버에서 클라이언트로 “Hello, client “ 메시지 전송

서버에는 클라이언트의 IP 주소 출력

소스코드 (ex01.c)

```
#define BACKLOG 10

int main(void)
{
    int sockfd, new_fd;
    struct sockaddr_in server_addr;
    struct sockaddr_in client_addr;
    int sin_size;

    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket");
        exit(1);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(60000);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    memset(&(server_addr.sin_zero), '\0', 8);

    if(bind(sockfd, (struct sockaddr *)&server_addr, sizeof(struct
sockaddr)) == -1) {
        perror("bind");
        exit(1);
    }
    if(listen(sockfd, BACKLOG) == -1) {
        perror("listen");
        exit(1);
    }
}
```

```
while(1) {
    sin_size = sizeof(struct sockaddr_in);
    if((new_fd = accept(sockfd, (struct sockaddr *)
&client_addr, &sin_size)) == -1) {
        perror("accept");
        continue;
    }
    printf("server : got connection from %s \n",
inet_ntoa(client_addr.sin_addr));
    if(!fork()) {
        close(sockfd);

        if(send(new_fd, "Hello, client!\n", 14, 0) == -1)
            perror("send");
        close(new_fd);
        exit(0);
    }
    close(new_fd);
    while(waitpid(-1, NULL, WNOHANG) > 0);
}
return 0;
}
```

TCP 클라이언트 프로그램

- 앞서 작성한 TCP 서버 프로그램에 맞춰 클라이언트 프로그램을 작성해보자.

〈조건〉

서버 IP 주소 – 프로그램 실행시 서버의 호스명 인자로 전달

서비스 포트번호 – 60000번

서버가 보내는 메시지 출력

소스코드 (ex02.c)

```
#define MAXDATASIZE 100

int main(int argc, char *argv[])
{
    int sockfd, numbytes;
    socklen_t addr_len;
    char buf[MAXDATASIZE];
    struct hostent *he;
    struct sockaddr_in server_addr;

    if(argc != 2) {
        fprintf(stderr, "usage : client hostname \n");
        exit(1);
    }
    if((he = gethostbyname(argv[1])) == NULL) {
        perror("gethostbyname");
        exit(1);
    }
    if((sockfd = socket(AF_INET, SOCK_STREAM, 0))
    == -1) {
        perror("socket");
        exit(1);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(60000);
    server_addr.sin_addr = *((struct in_addr *)he-
    >h_addr);
    printf("[ %s ]\n", (char*)
    inet_ntoa(server_addr.sin_addr));

    memset(&(server_addr.sin_zero), '\0', 8);
    if(connect(sockfd, (struct sockaddr *)&server_addr,
    sizeof(struct sockaddr)) == -1) {
        perror("connect");
        exit(1);
    }
    if((numbytes = recv(sockfd, buf, MAXDATASIZE-1,
    0)) == -1) {
        perror("recv");
        exit(1);
    }
    buf[numbytes] = '\0';
    printf("Received : %s\n", buf);
    close(sockfd);
    return 0;
}
```


리눅스 네트워크 프로그래밍

12장 UDP 프로그래밍

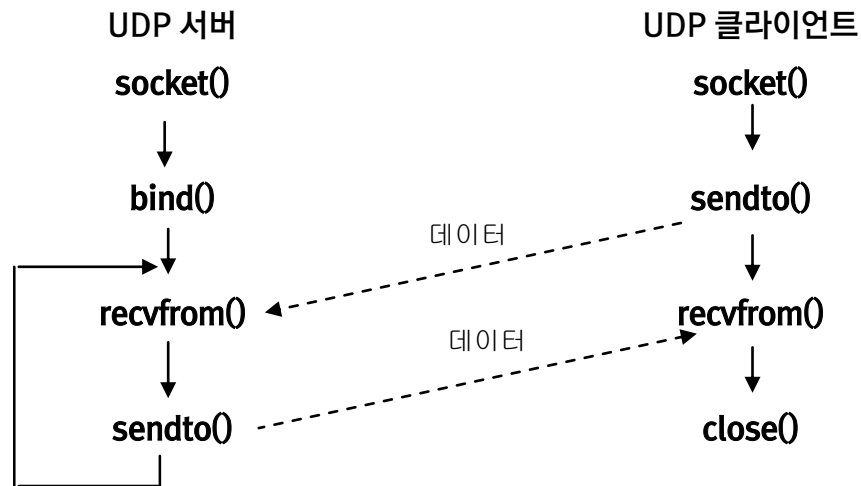


목차

1. UDP 서버/클라이언트 구조
2. UDP 프로그램을 위한 소켓 API
3. UDP 서버/클라이언트 프로그램

UDP 서버와 클라이언트 구조

- UDP는 connectionless 프로토콜 이므로 복잡한 연결 절차 없이 서버가 동작 중이면 언제든지 데이터를 송수신 할 수 있다. 통신 구조가 간단하기 때문에 속도를 요하는 음성이나 화상 데이터에 많이 사용되며 일회성 응답 서비스 등에도 많이 사용되고 있다. 이 내용을 소켓 API 와 함께 일련의 흐름으로 나타내보면 다음과 같다.



UDP 서버와 클라이언트 함수 호출 순서

UDP 소켓 API

■ 소켓 생성

- 앞서 TCP 소켓과 마찬가지로 소켓을 사용하기 위해서는 먼저 `socket()` 함수를 이용하여 소켓을 생성해야 한다. 함수의 원형은 TCP 소켓에서와 동일하다. 다시 한번 반복해서 살펴보자.

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int socket (int domain, int type, int protocol);
```

- 첫 번째 인자 `domain`에 지정할 수 있는 도메인의 종류는 다음과 같다.

도메인 종류	인터페이스
AF_UNIX	유닉스 도메인 소켓 (로컬 통신)
AF_INET	IPv4 네트워크 도메인 소켓
AF_INET6	IPv6 네트워크 도메인 소켓

- 두 번째 인자인 `type`에는 소켓 유형을 지정하는데 TCP 소켓의 경우 스트림 타입을 지정했지만 UDP 소켓의 경우에는 데이터그램 유형을 지정해야 한다.

소켓 유형	관련 프로토콜
SOCK_STREAM	스트림 타입, TCP
SOCK_DGRAM	데이터그램 타입, UDP
SOCK_RAW	RAW 타입, IP

UDP 소켓 API

- 세 번째 인수 protocol에는 프로토콜의 번호를 지정하도록 되어있으나 일반적으로 0을 설정한다.
- 이 함수 호출이 성공하면 소켓 디스크립터를 반환한다. 실패하면 -1 을 반환한다.

- UDP 소켓 생성에 관한 예제 코드를 살펴보자

```
int sockfd;  
  
sockfd=socket(AF_INET, SOCK_DGRAM, 0);  
if(sockfd==-1) {  
    perror( "socket" );  
    exit(1);  
}
```

- 위의 예에서 도메인을 INET 으로 소켓 유형은 DGRAM , 즉 udp 기반으로 소켓을 생성 했습니다. 성공한 경우에는 소켓 디스크립터를 반환하는데 이는 리눅스의 프로세스 구조에서 살펴봤던 파일 디스크립터이다. 따라서 파일 디스크립터 관련 함수를 소켓에 적용할 수도 있다.

UDP 소켓 API

■ 주소 설정

- UDP 에서도 TCP 와 마찬가지로 클라이언트에서 서버로 연결 요청 시 서버의 특정 소켓에서 응답을 하려면 IP 주소와 포트 번호를 설정해야 한다. 이때 사용되는 함수가 bind() 함수로 함수의 원형은 다음과 같다.

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *my_addr, socklen_t addrlen);
```

- 첫 번째 인자 sockfd에는 socket() 함수의 반환 값인 소켓 디스크립터를 지정한다.
- 두 번째 인자 myaddr에는 소켓 주소의 정보를 가리키는 주소를 지정한다.여기서 소켓 주소의 구조체 자료형은 소켓 생성시 선택한 도메인의 종류에 따라 아래와 같이 결정된다

도메인 종류	소켓 구조체 자료형
AF_UNIX	struct sockaddr_un
AF_INET	struct sockaddr_in
AF_INET6	struct sockaddr_in6

- 세 번째 인자 addrlen은 소켓 주소의 길이를 지정한다.
- 이 함수의 호출이 성공하면 0을 반환하고, 실패하면 -1을 반환한다.

UDP 소켓 API

- 다음은 bind 함수의 예제 코드이다.

```
struct sockaddr_in myaddr;  
my_addr.sin_family = AF_INET;  
my_addr.sin_port = htons(60000);  
my_addr.sin_addr.s_addr = htonl(INADDR_ANY);  
memset(&(my_addr.sin_zero), '\0', 8);  
  
if(bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr)) == -1)  
{  
    perror("bind");  
    exit(1);  
}
```

- 위의 예에서 IPv4 용의 소켓 주소 자료형인 sockaddr_in 의 변수 myaddr 에 도메인 종류, 포트번호, IP 주소등을 대입했다. 여기서 IP 주소에 INADDR_ANY란 0.0.0.0 으로 정의되어 있는 값으로 이렇게 설정하면 소스 코드 수정 없이 어디서든 수행시킬 수 있다. 이 값을 두번째 인자로 전달할 때 범용 자료형인 (struct sockaddr *) 으로 캐스팅하여 전달했다.

UDP 소켓 API

- 데이터 송수신
- UDP 소켓을 이용하여 데이터를 송수신하기 위하여 `sendto()` 또는 `recvfrom()` 함수를 이용한다. 이 두 함수의 원형을 살펴보자.

```
#include <sys/types.h>
#include <sys/socket.h>

int sendto(int sockfd, const void *buf, size_t len, int flag,
           const struct sockaddr *to, socklen_t tolen);

int recvfrom(int sockfd, const void *buf, size_t len, int flag,
             const struct sockaddr *from, socklen_t *fromlen);
```

- 네 번째 인수까지는 `send()`, `recv()` 함수와 동일하므로 다섯 번째와 여섯 번째 인수에 대해서만 살펴보자.
- `sendto()` 함수의 다섯번째 인자 `to`에는 데이터를 전송할 소켓 주소 변수의 포인터를 ,
- 여섯번째 인자 `tolen`에는 소켓 주소의 길이를 전달한다.
- `recvfrom()` 함수의 다섯 번째 인자 `from`에는 데이터를 보낸 상대방의 소켓 주소를 받아오고, 여섯 번째 인자 `fromlen`에는 상대방 소켓 주소의 크기를 받아온다.
- 이 두 함수가 성공적으로 호출되면 실지로 전송, 혹은 수신한 바이트 수를 반환하며, 실패하면 `-1`을 반환한다. `errno` 변수를 확인하면 오류의 원인을 알 수 있다

UDP 소켓 API

- 데이터 송수신 관련 예제 코드를 살펴보자

```
while(1) {  
    . . .  
    if(sendto(sockfd, buf, strlen(buf), 0,  
        (struct sockaddr*) &to_addr, sizeof(to_addr)) == -1) {  
        perror( "sendto" );  
        break;  
    }  
    if((numbytes = recvfrom(sockfd, buf, MAXDATASIZE-1, 0,  
        (struct sockaddr*) &from_addr, &sock_len)) == -1) {  
        perror("recvfrom");  
        break;  
    }  
    . . .  
}
```

- 위의 예제 프로그램은 sendto() 함수를 이용하여 to_addr에 지정된 목적지로 buf 내용을 전송하고, 다시 recvfrom() 함수를 통해 buf 로 데이터를 수신 받는 것과 동시에 from_addr 에 발송지의 주소를 받아오는 부분이다. UDP 소켓을 이용하여 데이터를 송수신할 때는 송수신할 때마다 소켓의 주소를 명시적으로 표현하는 sendto(), recvfrom() 함수를 이용한다.

UDP 소켓 API

■ 연결 해제

- TCP 와 마찬가지로 UDP 소켓의 연결을 해제하기 위해서도 `close()` 나 `shutdown()` 함수를 이용한다.

```
int close(int sd);  
int shutdown(int sd, int how);
```

- `close` 함수는 일반 파일을 해제하듯이 소켓 디스크립터를 전달해 해제한다. 이 함수를 호출하면 해당 프로세스와 소켓과의 연결만 해제되고 이 소켓을 열어놓은 다른 프로세스가 있는 경우 모든 프로세스가 해제를 해야 실지로 소켓이 종료 처리된다.
- 이에 반해 `shutdown` 함수는 즉시 소켓을 종료하므로 이 소켓을 열어놓은 프로세스에게 영향을 미치게 된다. 따라서 일반적으로는 `close` 함수로 해제를 한다.
- `shutdown()` 함수의 두번째 인자 `how` 에 지정할 수 있는 값은 다음과 같다.

how	소켓 구조체 자료형
SHUT_RD	읽기 채널만 닫기
SHUT_WR	쓰기 채널만 닫기
SHUT_RDWR	소켓 즉시 종료

UDP 서버/클라이언트 프로그램

- 1.UDP 서버 프로그램
- 2.UDP 클라이언트 프로그램

UDP 서버 프로그램

- 앞서 학습한 UDP 소켓 API 를 이용하여 다음과 같은 조건의 UDP 서버 프로그램을 작성해보자.

〈조건〉

소켓 주소 – IP 주소(0.0.0.0)

포트번호(60000)

클라이언트에서 보내는 데이터를 클라이언트 IP주소와 함께 화면에 출력



앞서 작성한 TCP 서버프로그램(ex01.c)를 복사하고, “quit” 가 들어올 때까지 클라이언트에서 전송하는 메시지를 수신 받아 화면에 출력하는 UDP 서버 프로그램으로 변경해 보자.

UDP 클라이언트 프로그램

- 앞서 작성한 UDP 서버 프로그램에 맞춰 클라이언트 프로그램을 작성해보자.

〈조건〉

서버 IP 주소 – 프로그램 실행시 서버의 호스명 인자로 전달

사용자가 입력한 메시지 서버로 전송



실행 결과

```
user@ubuntu: ~/online_lsp/ch21
```

```
user@ubuntu:~/online_lsp/ch21$ ./ex01  
[ 192.168.230.149 ] "hello, udp server?"  
[ 192.168.230.149 ] "this is udp client"  
[ 192.168.230.149 ] "have a good day~~"
```

```
user@ubuntu: ~/online_lsp/ch21
```

```
user@ubuntu:~/online_lsp/ch21$ gcc ex02.c -o ex02  
user@ubuntu:~/online_lsp/ch21$ ./ex02 ubuntu  
send string? hello, udp server?  
sent 18 bytes to 192.168.230.149  
user@ubuntu:~/online_lsp/ch21$ ./ex02 ubuntu  
send string? this is udp client  
sent 18 bytes to 192.168.230.149  
user@ubuntu:~/online_lsp/ch21$ ./ex02 ubuntu  
send string? have a good day~~  
sent 17 bytes to 192.168.230.149  
user@ubuntu:~/online_lsp/ch21$
```


exercise.c 는 udp 클라이언트 프로그램이다. 서버 ex01 에게 문자열을 10번 전송하는 프로그램이다. 학습 과정에서 살펴본 함수 호출순서와 소켓 API 함수의 원형을 이용하여 소스코드의 빈 칸을 완성하시오.

소스코드 (exercise.c)

```
int main(int argc, char* argv[])
{
    int sockfd, i;
    struct sockaddr_in server_addr;
    int numbytes;
    char buf[80];

    if(argc!=2) {
        fprintf(stderr, "Usage : ./exercise IP주소\n");
        exit(1);
    }

    if((sockfd = socket(AF_INET, 1, 0)) == -
1){
        perror("socket");
        exit(1);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(60000);
    server_addr.sin_addr.s_addr = inet_addr(argv[1]);
    memset(&(server_addr.sin_zero), '\0', 8);
```

```
    for(i=0; i<10; i++) {
        sprintf(buf, "Hello Server %d", i);
        if((numbytes = 2 (sockfd, buf, strlen(buf), 0,
(struct sockaddr *) 3 sizeof(struct
sockaddr))) == -1) {
            perror("sendto");
            exit(1);
        }
        printf("sent %d bytes to %s\n", numbytes,
inet_ntoa(server_addr.sin_addr));
    }
    close(sockfd);
    return 0;
}
```

실행 결과

```
user@ubuntu: ~/online_lsp/ch21
user@ubuntu:~/online_lsp/ch21$ ./ex01
[ 192.168.230.149 ] "Hello, Server 0"
[ 192.168.230.149 ] "Hello, Server 1"
[ 192.168.230.149 ] "Hello, Server 2"
[ 192.168.230.149 ] "Hello, Server 3"
[ 192.168.230.149 ] "Hello, Server 4"
[ 192.168.230.149 ] "Hello, Server 5"
[ 192.168.230.149 ] "Hello, Server 6"
[ 192.168.230.149 ] "Hello, Server 7"
[ 192.168.230.149 ] "Hello, Server 8"
[ 192.168.230.149 ] "Hello, Server 9"
```

```
user@ubuntu: ~/online_lsp/ch21
user@ubuntu:~/online_lsp/ch21$ ./exercise 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
sent 15 bytes to 192.168.230.149
user@ubuntu:~/online_lsp/ch21$
```

리눅스 네트워크 프로그래밍

13장 입출력 다중화



목차

1. 입출력 다중화
2. 전형적인 I/O Multiplexing 함수
3. 고성능 I/O Multiplexing

입출력 다중화

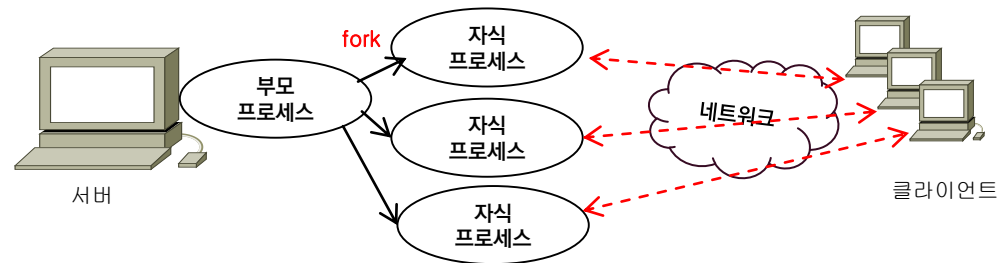
1. 입출력 모드
2. 다중적인 입출력 해결

입출력 모드

- 일반적으로 네트워크 에서 입출력 방식은 다음과 같은 모드 중 하나로 이루어진다.
- 블로킹 (Blocking) 모드
 - 소켓의 기본 모드로 네트워크로부터 데이터를 수신하기 위해 `recv()` 함수를 호출한 경우 상대방으로부터 데이터가 수신되지 않으면 해당 프로세스는 수행을 멈추고 기다린다. 이런 블록 현상은 각종 입출력 함수에서 발생할 수 있으며 시스템 또는 네트워크의 어떤 조건이 만족할 때까지 기다리게 된다.
- 년-블로킹 (Non-Blocking) 모드
 - 년-블로킹 모드는 입출력 함수를 이용하여 데이터를 송수신할 수 있으면 즉시 실행하고, 할 수 없는 경우에도 함수 안에서 블로킹되지 않고 즉시 빠져 나와 다른 작업을 수행할 수 있게 해주는 모드를 말한다. 물론 반환 값을 이용하여 오류 발생 여부를 확인할 수 있다.
- 다중화된 I/O 모드
 - 입출력이 동시에 다중적으로 혹은 연속적으로 일어나는 경우로 시스템에서 이벤트를 감지할 때마다 입출력 함수를 수행하도록 하는 입출력 방법이다.

다중적인 입출력 해결

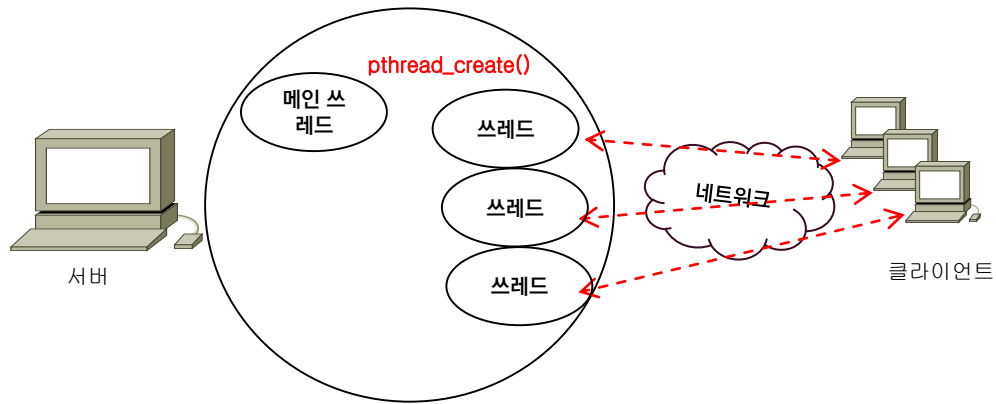
- 다중적인 입출력이 일어나는 구조의 네트워크 프로그램은 다음과 같은 방법들로 해결할 수 있다.
- 멀티 프로세스
 - 클라이언트 요청이 있을 때마다 fork() 시스템 콜을 호출하여 동시에 서비스가 이루어지도록 구성하는 방식



멀티 프로세스 방식의 입출력 다중화

다중적인 입출력 해결

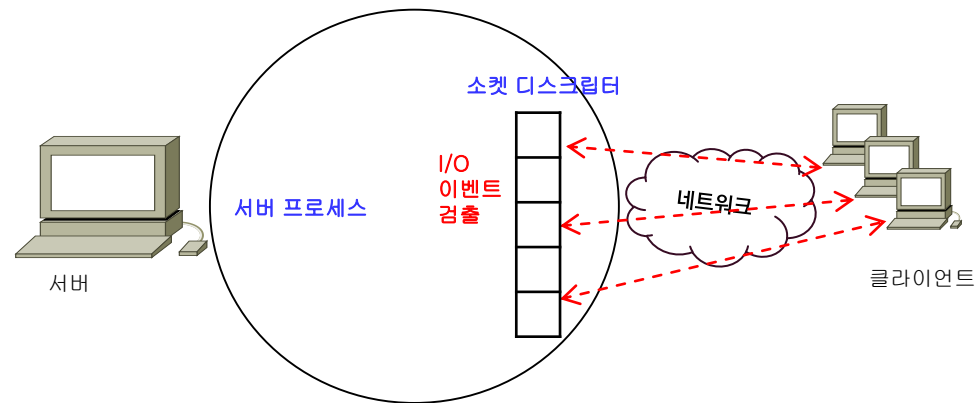
- 멀티 쓰레드
 - 클라이언트 요청이 있을 때마다 `pthread_create()` 함수 호출하여 동시에 서비스가 이루어지도록 구성하는 방식



멀티 쓰레드 방식의 입출력 다중화

■ IO Multiplexing 함수

- 각 소켓의 I/O 이벤트를 발생을 검출하여 서비스하는 방식으로 별도의 프로세스나 스레드를 이용하지 않기 때문에 메모리 사용률을 낮추고 문맥교환에도 영향을 주지 않기 때문에 시스템 성능을 높일 수 있다.



IO Multiplexing 방식의 입출력 다중화

- 지금까지 살펴본 다중적인 입출력 구조의 네트워크 프로그램을 가장 효율적으로 해결하는 방식은 IO Multiplexing 함수를 이용하는 것이다. 이제부터 그 방법에 대해 알아보기로 하자

전통적인 IO MULTIPLEXING 함수

- 1.select 방식
- 2.polling 방식

SELECT 방식

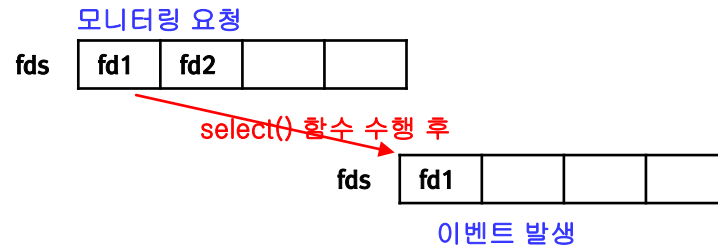
- select 방식이란 데이터의 송수신을 위해 직접 입출력 함수를 호출하기에 앞서 어떤 소켓에 입출력 이벤트가 일어났는지를 모니터링하여 해당 소켓으로부터 데이터를 송수신하는 방법을 의미한다. 이때 사용되는 함수는 select() 함수로 다음과 같은 원형을 가지고 있다.

```
#include <unistd.h>
#include <fcntl.h>

int select(int maxfd, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *tvptr);
```

- 여기서 첫 번째 인자인 maxfd에는 최대 파일(소켓) 디스크립터의 번호+ 1을 지정한다.
- 두 번째 인자 readfds에는 읽기 이벤트를 모니터링할 파일 디스크립터 집합의 주소를
- 세 번째 인자 writefds 에는 쓰기 이벤트를 모니터링할 파일 디스크립터 집합의 의 주소를
- 네 번째 인자 exceptfds에는 예외(오류) 이벤트를 모니터링할 파일 디스크립터 집합의 주소를 지정한다. 두번째부터 네번째 인자 사용시 주의 할 점은 이 함수를 빠져나올 때 실제로 이벤트 발생한 디스크립터를 인자로 다시 되돌려준다는 점다. 즉, 호출 시에 전달된 값이 함수가 종료된 이후에 유지되지 않는다는 점이다. 예를 들어 다음과 같다.

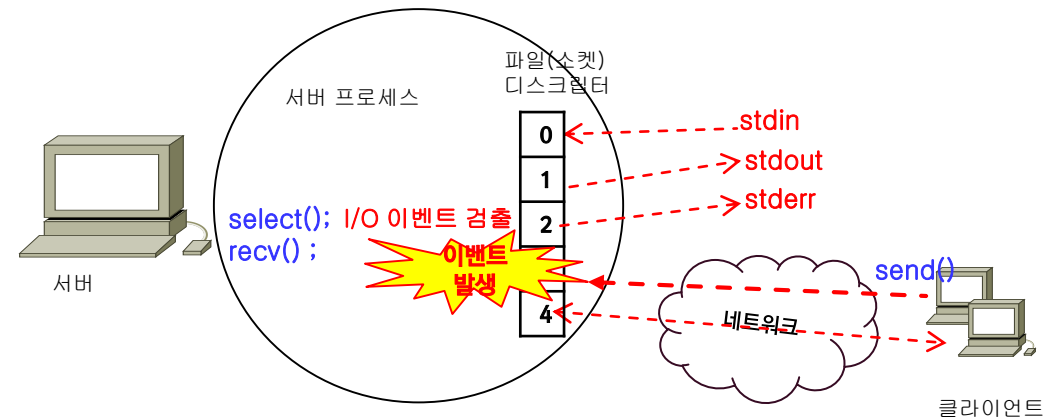
SELECT 방식



- 다섯번째 인수 tvptr 에는 select() 함수의 타임아웃 값에 대한 주소를 지정한다. NULL을 지정하면 이벤트가 발생할 때까지 계속 모니터링 한다.
- 이 함수가 성공하면 이벤트가 발생한 파일 디스크립터의 개수를 반환한다. 실패하면 -1을 반환한다.
-

SELECT 방식

- 다음 그림은 select 함수의 동작을 알기 쉽게 그림으로 표현한 것이다.



select 함수의 동작 원리

SELECT 방식

- select() 함수와 함께 사용되는 매크로 함수들이 있는데 이 함수들은 파일 디스크립터의 모임인 fd_set 자료형의 파일 디스크립터 집합에 사용한다.

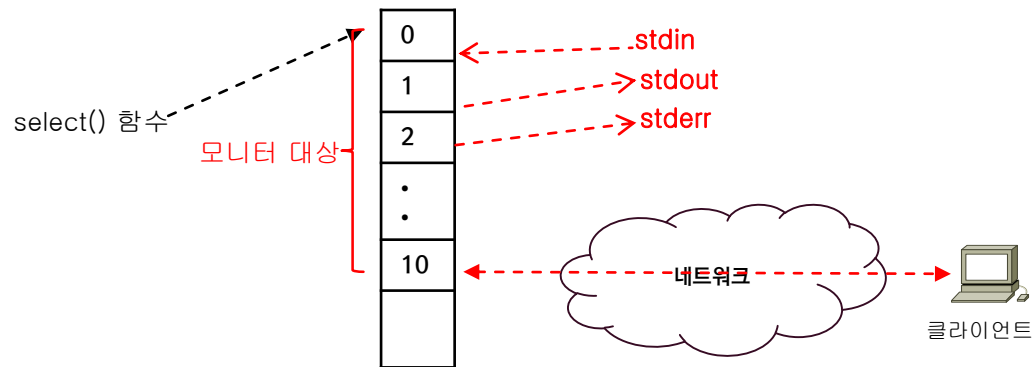
매크로	의 미
FD_ZERO(fd_set *fdset)	fdset 변수의 초기화
FD_SET(int fd, fd_set *fdset)	fdset 변수에 fd 등록
FD_CLR(int fd, fd_set *fdset)	fdset 변수로부터 fd 삭제
FD_ISSET(int fd, fd_set *fdset)	fdset 중 fd가 설정되어있는지 확인 select() 함수에서 벗어나 어떤 파일 디스크립터에 이벤트가 일어났는지 확인할 때 사용



TCP 기반으로 일대일 채팅 프로그램 ex01.c, ex02.c 을 작성하고 있다. select() 를 이용하여 chatting() 함수를 완성해보자.

POLL 방식

- 앞서 살펴본 `select()` 함수는 이벤트 감지를 위해 파일 디스크립터 0번부터 최대 파일 디스크립터까지를 모니터하는 함수다.



select 함수의 모니터링

- 위 그림에서 실제로 이벤트 감지 대상의 디스크립터는 10 번 하나일지라도 `select` 함수는 0번부터 순차적으로 모니터링 한다. 만약 최대 파일디스크립터가 100번이라면 `select`는 0번부터 100번까지를 일일이 이벤트 발생 여부를 확인한다는 의미다. 따라서 파일 디스크립터 10번에 이벤트 발생이 되었는데 `select` 함수가 0번을 확인 중이었다면 실제 이벤트를 감지하는 시간은 좀 늦어지게 된다. 물론 사람들이 느끼기에 이 시간 차이는 미미한 수준이지만 CPU 에서 체감하는 시간 차이는 아주 클 수도 있다.

POLL 방식

- poll() 함수는 모니터링하고 싶은 디스크립터와 이벤트의 종류를 미리 등록해 놓고 등록된 디스크립터만 모니터링하는 함수다. 함수의 원형은 다음과 같다.

```
#include <poll.h>

int poll(struct pollfd *ufds, unsigned int nfds, int timeout);
```

- 여기서 첫번째 인수인 ufds에는 이벤트 감지를 원하는 파일 디스크립터에 관한 정보를 저장한다, 이때 구조체 자료형은 다음과 같다.

```
struct pollfd {
    int fd;           // 감지를 원하는 파일 디스크립터
    short events;     // 감지할 이벤트 종류
    short revents;    // 실제로 발생한 이벤트
};
```

- 이벤트의 종류는 다음과 같다.

events 종류	의미
POLLIN	수신 데이터
POLLPRI	긴급하게 읽을 데이터
POLLOUT	송신 데이터
POLLERR	오류 발생
POLLHUP	연결이 끊어짐
POLLNVAL	잘못된 요청

POLL 방식

- 여러 종류의 이벤트를 지정해야 할 때에는 events 멤버에 비트 OR 연산자인 | 를 이용하여 추가할 수 있다. revents 멤버는 실제로 발생한 이벤트를 받아오는 멤버다,

두번째 인자인 nfds 는 이벤트 검출을 원하는 파일 디스크립터의 개수를 지정하고,

세번째 인자인 timeout은 select 함수의 time out과 유사한 의미를 갖는다.

timeout이 -1인 경우 이벤트가 검출될 때까지 블록,

0인 경우 이벤트 발생한 파일 디스크립터 확인만하고 즉시 반환,

양수인 경우 지정된 초 수만큼 이벤트 모니터링.

- 반환 값이 -1인 경우는 함수가 실패한 경우이고 0인 경우는 이벤트 발생한 파일 디스크립터 없이 타임아웃 된 경우, 양수인 경우는 이벤트가 감지된 파일 디스크립터의 개수다.



ex03.c 와 ex04.c 소스에서 poll() 함수의 사용을 확인하고 실행하면서 select() 함수와의 차이점을 생각해보자.

고성능 IO MULTIPLEXING

- 1.select와 poll 방식의 문제점
- 2.epoll 함수의 이해와 사용

SELECT 와 POLL 방식의 문제점

- 앞서 select 나 poll 함수를 이용한 IO Multiplexing 방법은 동시 접속 수가 수 십 개 정도일 때는 문제가 없지만 그 수를 넘어가면 성능 면에서 문제가 일어나기 시작한다.
- select() 함수
- select 함수는 프로세스 내의 모든 파일 디스크립터를 대상으로 하는 함수로 사용 예를 보면 다음과 같다.

```
while(1){  
    read_fds = mask;  
    select(sd+1, &read_fds, NULL, NULL, NULL);  
    . . .  
}
```

- select 함수의 첫번째 인자는 파일(소켓) 디스크립터의 개수로 0번 부터 이 개수만큼 이벤트가 발생했는지 각 파일 디스크립터를 모니터링한다. 만약 파일 디스크립터 3번과 78번, 이 두 가지 디스크립터만 확인하는 경우에도 select 함수에서는 0번부터 78번까지 모두 모니터링한다. 따라서 실제 이벤트 발생을 감지하는 시간이 늦어질 수 있다. 오픈 되어있는 파일 또는 소켓이 수십 개 정도라면 감지 시간의 지연이 그리 크지 않지만 수 백 개라면 지연시간은 상당히 길어질 확률이 높다.
- select 함수의 또 한 가지 문제점은 이 함수를 호출하기에 앞서 항상 감지를 요청하는 파일 디스크립터들을 새로 등록해야한다는 점이다. 그 이유는 호출할 때 전달하는 파일 디스크립터의 정보가 저장되지 않고 그 변수에 실제로 이벤트가 발생한 파일 디스크립터의 정보가 다시 저장되기 때문이다

SELECT 와 POLL 방식의 문제점

- poll() 함수
- poll() 함수는 이벤트를 감지하려는 파일 디스크립터만 모아서 모니터링하는 함수로 사용 예는 다음과 같다.

```
struct pollfd client_list[500];

while (1) {
    n = poll(client_list, maxi + 1, 1000);
    if (client_list[0].revents & POLLIN) {
        socklen = sizeof(client_addr);
        client_sockfd = accept(server_sockfd, (struct sockaddr *)&client_addr, &socklen);
        for (i = 1; i < MAX; i++)
            if (client_list[i].fd < 0) {
                client_list[i].fd = client_sockfd;
                client_list[i].events = POLLIN;
                break;
            }
    }
}
```

- 여기에서 client_list 배열이 이벤트 감지에 대한 정보를 저장하는 곳으로 수 십 개 정도의 파일 디스크립터에 대한 정보가 모여 있는 경우에는 select() 함수에 비해 성능이 좋겠지만 수 백 개의 파일 디스크립터를 모니터링하는 경우에는 select 함수와 큰 차이가 없다.

EPOLL 함수의 이해와 활용

1) epoll 함수의 이해

epoll 함수는 이벤트가 발생한 파일의 목록을 반환하는 함수로 반환된 정보를 이용하여 이벤트 처리를 하므로 앞서 살펴본 select()나 poll() 함수에 비해 빠른 속도로 이벤트를 처리할 수 있다. 그리고 select 함수처럼 매번 이벤트 감지를 요청할 파일 디스크립터를 새로 등록할 필요가 없다. 일단 등록하면 삭제하기 전까지 그 값을 유지하기 때문이다.

epoll 에서 다루는 이벤트 방식에는 다음과 같은 두 가지 방식이 있다.

- Level Trigger

소켓 버퍼에 데이터가 있는 동안 이벤트를 계속 발생시키는 방식, select() 나 poll() 과 같이 동작.

- Edge Trigger

소켓 버퍼의 데이터 유무와 상관 없이 실제 이벤트가 발생했을 때 1번만 발생, 소켓을 논 블로킹 모드로 설정해야 함.

EPOLL 함수의 이해와 활용

2) epoll 함수

- `epoll_create()`
- `epoll_create()`는 이벤트를 저장하기 위한 공간을 커널에 요청하는 함수로 원형은 다음과 같다.

```
#include <sys/epoll.h>
int epoll_create(int size);
```

- 여기서 `size`는 커널에 요청하는 공간의 크기이지만 정확히 `size`만큼만 확보되는 것이 아니라 커널이 대략 어느 정도의 공간을 만들어야 할지는 정해준다. 성공하면 파일 디스크립터를 반환한다. 만약 더 이상 사용하지 않을 거라면 `close()` 함수로 닫아준다.

`size`는 일반적으로 예상 최대 동접수x1.5 정도로 잡는다. 그러나 리눅스 커널 버전 2.6.8 이후에는 `size`는 큰 의미가 없다.

- 사용 예는 다음과 같다.

```
int efd,size;

size=10;
if((efd=epoll_create(size)) == -1) {
    perror( "epoll_create" );
    exit(1);
}
```


EPOLL 함수의 이해와 활용

▪ epoll_ctl()

- epoll_ctl() 함수는 이벤트 감지를 원하는 파일(소켓) 디스크립터와 이벤트의 종류를 등록하는 함수로 그 원형은 다음과 같다.

```
#include <sys/epoll.h>
```

```
int epoll_ctl(int efd, int op, int fd, struct epoll_event *event);
```

- 여기서 첫번째 인자 efd 는 epoll_create() 함수의 반환값이고,
- 두번째 인자 op 에는 명령을 기술한다. 명령의 종류는 다음과 같다.

명령 종류	의 미
EPOLL_CTL_ADD	파일 디스크립터를 이벤트 풀에 등록하는 명령
EPOLL_CTL_DEL	파일 디스크립터를 이벤트 풀에서 삭제하는 명령
EPOLL_CTL_MOD	파일 디스크립터에 대한 이벤트의 멤버를 변경하는 명령

- 세번째 인자는 이벤트 감지를 요청하는 파일 디스크립터 번호를,
- 마지막 인자에는 이벤트의 종류를 기술하는데 그 자료형인 struct epoll_event는 다음과 같다.

EPOLL 함수의 이해와 활용

```
typedef union epoll_data {
    void *ptr;
    int fd;
    __uint32_t u32; __uint64_t u64;
} epoll_data_t;

struct epoll_event {
    __uint32_t events;    // 이벤트 종류
    epoll_data_t data;    // 사용자의 데이터로 직접 설정
};
```

- epoll_data 구조체의 첫번째 멤버인 events에는 다음과 같은 종류의 이벤트를 기술할 수 있다.

이벤트 종류	의 미
EPOLLIN	수신 데이터
EPOLLOUT	전송 데이터
EPOLLERR	오류 발생
EPOLLPRI	OOB(Out OF Band) 데이터가 수신된 경우
EPOLLET	이벤트 트리거를 엷지 트리거로 설정
EPOLLONESHOT	이벤트가 한번 감지되면 더 이상 이벤트를 발생시키지 않음.

- 위의 이벤트 종류는 | (비트 OR 연산자)를 이용하여 추가할 수 있다

EPOLL 함수의 이해와 활용

- 두번째 멤버인 data는 공용체 자료형으로 파일 디스크립터와 같은 정보를 저장하는 용도로 사용한다. 다음은 epoll_ctl() 함수의 사용 예다.

```
struct epoll_event client_list;  
  
clientlist.events=EPOLL_IN;  
clientlist.data.fd=sockfd;  
epoll_ctl(efd, EPOLL_CTL_ADD, sockfd, &client_list);
```

- 위 예제는 sockfd 에 대한 수신 이벤트 모니터링을 등록하는 부분이다.

EPOLL 함수의 이해와 활용

■ epoll_wait()

- 실제 이벤트가 발생하는걸 기다리는 함수로 함수의 원형은 다음과 같다.

```
#include <sys/epoll.h>
int epoll_wait(int epfd, struct epoll_event * events, int maxevents, int timeout);
```

- 여기서 첫번째 인자 efd 는 epoll_create() 함수에서 반환 받은 값이고,
- 두번째 인자 events 는 epoll_event 구조체 자료형으로 실제로 이벤트가 발생한 파일 디스크립터에 대한 정보를 얻어온다.
- 세번째 인자는 두번째 인자에 받아들 수 있는 최대 이벤트 수를,
- 마지막 인자에는 타임아웃 값을 설정한다. 시간의 단위는 1/1000 초다. -1이 경우는 이벤트 발생될 때까지 무한 루프를 의미한다.
- 함수의 반환 값은 이벤트가 발생한 파일 디스크립터의 개수를 의미한다.

EPOLL 함수의 이해와 활용

- 사용 예

```
struct epoll_event client_list[5];

if ( epoll_wait(efd, client_list, 5, -1) < 0 ) {
    perror( "epoll_wait() error : %s\n" , strerror(errno));
    exit(-1);
}
```

- 위 예제는 epoll_wait 함수에서 기다리고 있다가 이벤트가 발생하면 client_list에는 실제로 이벤트가 발생한 파일 디스크립터의 정보를 받아온다. 이때 최대 5개까지 받아들 수 있다.
- 이제 epoll 함수들을 이용하여 echo 서버를 완성해 보자



ex05.c 를 이용하여 epoll() 함수에 대해 좀더 자세하게 알아보자. 클라이언트는 ex04.c 를 이용한다.

EPOLL 함수의 이해와 활용

3) 레벨 트리거와 엣지 트리거

- 앞서 ‘1) epoll 함수의 이해’ 에서 epoll 함수의 동작 방식에는 레벨 트리거 방식과 엣지 트리거 방식의 두 종류가 있다는 것을 학습했다. 이 두 가지 방식은 어떤 차이가 있는지 자세히 알아보고 구현 예제를 살펴보도록 하자.
- 수신 버퍼에 100바이트가 쌓여있다고 가정해보자. 만약 사용자 프로그램에서 20바이트를 읽어갔다면 아직 수신 버퍼에 80바이트가 남아 있는 상태다.
- 이 경우 레벨 트리거 방식에서는 계속 이벤트가 발생하는 반면 엣지 트리거 방식에서는 수신 버퍼에 데이터가 남아 있는 것과 는 무관하게 수신되었을 당시에만 이벤트가 발생되므로 남아있는 데이터를 사용자프로그램에서 읽어올 수 있도록 소켓을 넌 블록 모드로 설정해야 한다.
- 기본 동작이 레벨 트리거 방식이므로 앞서 살펴본 ex05.c 는 전형적인 레벨 트리거 방식의 echo 서버다. 이번에는 엣지 트리거 방식의 echo 서버 프로그램을 살펴보자. (ex06.c)

EPOLL 함수의 이해와 활용

4) select 와 poll 과 epoll 차이점

- select 나 poll 함수는 1980년대 중반에 생성된 함수로 함수 호출 시마다 이벤트 검출을 원하는 파일 디스크립터의 정보를 인자로 전달하고 함수가 수행되고 나면 어떤 디스크립터에서 이벤트가 일어났는지 하나하나 확인 해야 한다.
- 이에 비해 epoll 함수는 리눅스 커널 2.6 부터 지원되는 함수로 파일 디스크립터를 등록하는 함수와 이벤트 발생을 확인하는 함수가 서로 다르게 구성되어 있어 매번 파일 디스크립터의 장보를 전달할 필요가 없다. 그리고 이벤트 발생시 이벤트가 발생된 디스크립터에 대한 목록을 반환하므로 이 목록만 처리하면 된다. 따라서 성능 면에서 epoll 함수를 사용하는 것이 더 좋다.
- 한편 select 함수는 대부분의 운영체제에 존재하지만 epoll 은 리눅스에서만 사용할 수 있는 함수다. 따라서 범용적인 통신 프로그램을 위해서는 select 또는 poll 함수에 대한 이해도 필요하다.

실습

- 다음과 같은 실행 결과의 서버 프로그램을 완성해보자. 위쪽 화면은 서버이고 아래 쪽은 각각 클라이언트이며 각 클라이언트는 ex04 프로그램을 이용한다. 각 클라이언트에서 전송하는 메시지가 서버를 통해 상대 클라이언트로 전송되고 있다. epoll 함수를 이용하여 서버를 구현하는 경우 소스코드의 빈 칸에 들어가야 할 내용은?

```
user@ubuntu: ~/online_lsp/ch25
1
1
1
1
[ moon ] hi, sun!
[ moon ] hi, sun!read data [ sun ] hi, moon~
1

user@ubuntu: ~/online_lsp/ch24
user@ubuntu:~/online_lsp/ch24$ ./ex04 ubuntu
[ 192.168.230.162 ]
이 름 : moon
hi, sun!
[ sun ] hi, moon~
[ sun ] Today is my birthday.
Oh! Happy birthday to you~~
[ sun ] Thank you.

user@ubuntu: ~/online_lsp/ch24
user@ubuntu:~/online_lsp/ch24$ ./ex04 ubuntu
[ 192.168.230.162 ]
이 름 : sun
[ moon ] hi, sun!
hi, moon~
Today is my birthday.
[ moon ] Oh! Happy birthday to you~~
Thank you.
```

소스코드 (exercise.c)

```

. . .
client_list = (struct epoll_event
*)malloc(sizeof(*client_list) * MAXCLIENT);

if (1 100) < 0) {
    perror("epoll_create");
    exit(1);
}
. . .
listen(sockfd, 5);
ev.events = EPOLLIN;
ev.data.fd = sockfd;
2
while(1) {
    if((n = 3
1)) == -1) {
        perror("epoll_wait");
    }
    for (i = 0; i < n; i++) {
        if (client_list[i].data.fd == sockfd) {
            printf("Accept\n");
            client_sd = accept(sockfd, (struct sockaddr
*)&clientaddr, &socklen);
            printf("server : got connection from %s \n",
inet_ntoa(clientaddr.sin_addr));
            ev.events = EPOLLIN;
            ev.data.fd = client_sd;
            epoll_ctl(efd, EPOLL_CTL_ADD, client_sd, &ev);
            if(count < MAXCLIENT)
                cust[count++].socksd = client_sd;
        } else {

```

```

        readn = read(client_list[i].data.fd, buf, 255);
        buf[readn] = '\0';
        if(readn <= 0) {
            4
            close(client_list[i].data.fd);
            printf("Close fd\n");
        } else {
            printf("read data %s\n", buf);
            int j;
            for(j=0; j<MAXCLIENT; j++) {
                if(cust[j].socksd == -1) continue;
                if(cust[j].socksd == client_list[i].data.fd) continue;
                write(cust[j].socksd, buf, readn);
            }
        }
    } // else-end
} // for-end
} // while -end
return 0;
}

```